

Implementación de una Plataforma de software web y móvil híbrida para generar bitácora de vuelo y hacer seguimiento de las horas de vuelo mensuales, trimestrales, semestrales y anuales de los pilotos de la Aerolínea Latam en Colombia.

Emmanuel Londoño Gómez

UNIVERSIDAD CATÓLICA DE ORIENTE

FACULTAD INGENIERIA DE SISTEMAS

PROGRAMA INGENIERÍA DE SISTEMAS

RIONEGRO

2026

Implementación de una Plataforma de software web y móvil híbrida para generar bitácora de vuelo y hacer seguimiento de las horas de vuelo mensuales, trimestrales, semestrales y anuales de los pilotos de la Aerolínea Latam en Colombia.

Autor

Emmanuel Londoño Gomez

Título profesional

Asesor:

Wider Farid Sánchez Garzón

UNIVERSIDAD CATÓLICA DE ORIENTE

FACULTAD INGENIERIA DE SISTEMAS

PROGRAMA INGENIERÍA DE SISTEMAS

RIONEGRO

2026

Notas de aceptación

Firma del presidente del jurado.

Firma del jurado

Firma del jurado

Rionegro viernes 20 de marzo de 2026

Dedicatoria

Este proyecto está dedicado a todas las personas que me han acompañado en mi camino formativo. A mi familia, cuyo respaldo cálido y constante ha sido clave para continuar y a mis compañeros que ofrecieron su conocimiento, apoyo y motivación en los momentos más retadores.

Reconozco también a quienes me orientaron académicamente, especialmente a los docentes, que compartieron no solo conocimiento sino también su tiempo y compromiso. A la universidad, por brindarme un entorno que me permitió crecer, desde quienes me guiaron hasta quienes me dieron una palabra de aliento.

Agradecimientos

Primero que todo a Dios por el don de la vida y de la familia.

Quiero expresar mi agradecimiento al alma mater que me abrió las puertas desde el primer día de clases para la formación profesional y personal. En especial, al profesor Wider Farid Sánchez Garzón, por su entrega, mentoría, metodología, estrategia, guía cercana y capacidad de impulsarme siempre hacia una versión más crítica y sólida de mi trabajo con énfasis en cómo se debe trabajar en un entorno laboral actualmente.

A la docente Luz Mery Ríos Alzate, por su acompañamiento, guía, conocimiento, orientación y experiencia a lo largo del programa.

Al comité académico, el cual proporcionó tiempo y valiosas opiniones de apoyo para enriquecer esta propuesta.

A mi familia, por su confianza, esperanza y paciencia. A mis compañeros y amigos por los aportes que me ayudaron a mejorar.

Un reconocimiento muy especial a Esteban Agudelo Flórez compañero de las largas jornadas de estudio y trabajo, especialmente por su alto sentido de responsabilidad y apoyo en la realización de mis estudios y trabajo de grado, resaltando especialmente el alto grado de tolerancia que ha tenido conmigo.

Resumen

Actualmente los pilotos de las aerolíneas suelen registrar sus horas de vuelo de forma manual, ya sea en hojas de cálculo o con ayuda de terceros, lo que puede generar información poco confiable y dificultar el cumplimiento de los requerimientos aeronáuticos. Estas situaciones pueden ocasionar problemas en el control de las horas de vuelo así como riesgos legales y operativos relacionados con el incumplimiento de los límites establecidos por las autoridades aeronáuticas.

Este trabajo presenta FlightHours, una plataforma web y móvil orientada a apoyar a los pilotos en el registro y seguimiento de sus horas de vuelo. Esta herramienta permite organizar la información de manera automática y realizar un control continuo de las horas acumuladas en distintos períodos de tiempo.

Los resultados muestran que la plataforma facilita el registro de los vuelos, reduce el esfuerzo necesario para registrar la bitácora y mejora el control de las horas de vuelo. Además, permite identificar y evitar con alertas, posibles incumplimientos, lo que contribuye al cumplimiento de la normativa y al bienestar del piloto.

Palabras clave: horas de vuelo, bitácora de vuelo, pilotos, control de horas, cumplimiento normativo, alertas aeronáuticas.

Abstract

Currently, airline pilots often record their flight hours manually either in spreadsheets or by relying on third parties which can generate unreliable information and make it difficult to comply with aeronautical requirements. These situations can lead to problems in tracking flight hours as well as legal and operational risks related to non-compliance with the limits established by aviation authorities.

This study presents FlightHours, a web and mobile platform aimed at supporting pilots in recording and tracking their flight hours. This tool allows information to be organized automatically and enables continuous monitoring of accumulated hours over different periods of time.

The results show that the platform facilitates flight recording, reduces the effort required to fill in the logbook and improves control over flight hours. Additionally, it allows identifying and preventing potential non-compliance through alerts, contributing to regulatory compliance and pilots' well-being.

Keywords: flight hours, flight logbook, pilots, flight hour tracking, regulatory compliance, aeronautical alerts.

CONTENIDO

CONTENIDO	8
1.ANTECEDENTES	10
2. PLANTEAMIENTO DEL PROBLEMA	12
3.JUSTIFICACIÓN	14
4. OBJETIVOS	16
4.1 Generales	16
4.2 Específicos	17
5.MARCO TEÓRICO	17
5.1 Prevenciones de fatiga	17
5.2 Número de aterrizajes	18
5.3 Aeronáutica - Bitácora de Vuelo	18
6.DISEÑO METODOLÓGICO	20
6.1 Fase 1: Diseño estratégico — El Qué	20
6.1.2 Visión: ¿Hasta dónde vamos a llegar?	21
6.1.3 Mapa de impacto	21
6.1.4 Modelo de Dominio	22
6.1.4.1 Modelo enriquecido	23
6.1.5 Event storming	24
6.1.6 Dimensionamiento del producto	25
6.1.7 Mapa de historias de usuario	25
6.1.8 Product backlog	26
6.1.9 Escritura de historias de usuario.	27
6.1.10 Plan de liberación	28
7.FASE 2.ARTEFACTOS TÉCNICOS	29
7.1 Diseño arquitectónico	29
7.1.1 Atributos de calidad	30
7.1.1.2 Escenarios de calidad	31
7.1.1.3 Funcionalidades críticas	32
7.1.1.4 Restricciones técnicas.	33
7.1.1.5 Restricciones de negocio.	34
7.1.2 Spikes	34
7.1.3 Alternativas de solución.	35
7.1.4 Arquetipo de solución.	35
7.1.5 Plataforma tecnológica.	36
7.2 Diseño detallado.	36
7.2.1 Diagrama de structs.	37
7.2.2 Diagrama de componentes.	38
7.2.3 Diagrama de paquetes.	40
7.2.4 Diagrama de secuencia.	40
7.2.5 Diagrama de despliegue.	41

7.2.6 Modelo entidad relación.	43
8. FASE 3.CONSTRUCCIÓN	44
8.1 Análisis estático	44
8.2 Devops	46
8.3 Pruebas de carga	49
9.RESULTADOS	50
9.1 Perspectiva del piloto	50
9.1.1 Inicio de sesión	51
9.1.2 Registro de nuevo piloto	52
9.1.3 Recuperación de contraseña	53
9.1.4 Autenticación	54
9.1.5 Modificar la contraseña	55
9.1.6 Perfil del piloto	56
9.1.7 Eliminar información del piloto	57
9.1.8 Consultar aerolínea	58
9.1.9 Consultar ruta aerolínea	59
9.1.10 Consultar bitácora diaria	60
9.1.11 Agregar bitácora	61
9.1.12 Consultar matrícula	62
9.1.13 Consultar modelo y tipo de Aeronave	63
9.1.14 Consultar los vuelos de una bitácora	64
9.1.15 Consultar la información del vuelo	65
9.1.16 Consultar las alertas	66
9.2 Perspectiva del administrador	67
9.2.1 Pantalla de inicio	67
9.2.2 Gestión de aerolíneas	68
9.2.3 Gestión de rutas de aerolínea	69
9.2.4 Gestión de rutas	70
9.2.5 Gestión de aeropuertos	71
9.2.6 Gestión de modelos de aeronave	72
9.2.7 Gestión de familias de aeronaves	73
9.2.8 Gestión de fabricantes	74
10. CONCLUSIONES	75
11. REFERENCIAS BIBLIOGRÁFICAS	76
12. GLOSARIO	78
13. LISTA DE ANEXOS	86

Tabla 1*Figuras presentadas en la sección de resultados del proyecto FlightHours*

Nº Figura	Nombre
Figura 1	Inicio de Sesión para los dos roles
Figura 2	Registro de un nuevo piloto
Figura 3	Recuperación de Contraseña
Figura 4	Autenticación
Figura 5	Modificar la contraseña
Figura 6	perfil del piloto
Figura 7	Eliminar informacion del piloto
Figura 8	Consultar aerolínea
Figura 9	Consultar Ruta Aerolínea por código iata de los aeropuertos
Figura 10	Consultar Bitácora Diaria del piloto.
Figura 11	Agregar bitácora con los datos que se piden.
Figura 12	Consultar matrícula, filtro por nombre
Figura 13	Consultar modelo y tipo aeronave, filtro por número de matrícula.
Figura 14	Consultar los vuelos de una bitácora
Figura 15	Consultar la información del vuelo
Figura 16	Editar y eliminar la información del vuelo
Figura 17	Consultar las alertas
Figura 18	Pantalla de inicio del administrador
Figura 19	Gestion de aerolíneas
Figura 20	Gestión de rutas de aerolínea para el piloto.
Figura 21	Gestion de ruta
Figura 22	Gestionar aeropuertos
Figura 23	Gestión de modelos de aeronave
Figura 24	Gestion de familia de aeronave
Figura 25	Gestión de Fabricantes.

1.ANTECEDENTES

Debido a que han habido accidentes aeronáuticos no solamente en Colombia, las tendencias generales dicen que ha habido uso ineficiente de simuladores que consisten en sesiones apresuradas con emergencias aisladas, reduciendo efectividad hasta 44% menos incidentes con entrenamiento realista y dificultad para simular fallos complejos como pérdida total de empuje o sobrevelocidad, sin cues físicos reales (simutech) y que el error humano causó el 70-90% de casos fatales (aviacioninec), los porcentajes clave con la fatiga fueron entre 15-20% (datos 2001-2012, NTSB) y 21-23 % en investigaciones mayores (opinion +1) junto con el dato de que en 2025, el porcentaje clave consiste en errores de tripulación en 25%, manejo manual en 24% e incumplimiento SOP en 20% (iata), teniendo en cuenta esos datos se encontraron con 5 aplicaciones que ayudan a las operaciones de vuelo

CrewLounge: Aplicación dirigida a pilotos y tripulantes de cabina. Permite llevar un seguimiento de horas de vuelo, gestionar horarios y más. Está disponible en varias plataformas, para más información sobre la aplicación ver anexo 14

ForeFlight: Aunque es principalmente una aplicación de planificación de vuelos, también ofrece funciones de registro de tiempo de vuelo. Es popular entre pilotos de aviación general y comercial. Está disponible en iOS, para más información sobre la aplicación ver anexo 15

FlightLogger: Es una aplicación basada en la web que proporciona una plataforma completa de seguimiento de tiempo de vuelo y administración de registros. Es compatible con varios dispositivos y sistemas operativos, para más información sobre la aplicación ver anexo 16.

LogTen Pro: Es una aplicación ampliamente utilizada que permite a los pilotos rastrear su tiempo de vuelo, planificar futuros vuelos, gestionar certificaciones y más. Está disponible tanto en plataformas iOS como en macOS, para más información sobre la aplicación ver anexo 17

ZuluLog: Aplicación de seguimiento de horas de vuelo con funciones de planificación y gestión de registros. Está diseñada para pilotos de aviación general y comercial, para más información sobre la aplicación ver anexo 18.

Todas estas aplicaciones están diseñadas para ayudar a los pilotos y a un nicho muy especial, en la gestión de su tiempo de vuelo y registros. Se puede ver que muchas se basan en brindar un servicio netamente a usuarios que tengan acceso a dispositivos IOS lo que conlleva a generar unas restricciones para el usuario.

2. PLANTEAMIENTO DEL PROBLEMA

Actualmente, los pilotos realizan seguimiento de sus horas de vuelo en papel, o le pagan a una persona para que realice el diligenciamiento de estas dependiendo del registro brindado por la empresa aeronáutica, lo que conlleva a que la información no sea fiable y tampoco cuentan con una herramienta que les genere un alertamiento de estar próximos a vulnerar los siguientes requisitos aéreos:

1. Cada tres meses aproximación de categoría: Aproximación con muy baja visibilidad, con muy buena aproximación a la pista y/o aproximación con piloto automático.

2. Cantidad de aterrizajes: Mínimo 3 en 30 días Limitación de alertas de vuelo (Estas son por normativa legal y pueden acarrear complicaciones legales como demandas a la aerolínea y/o piloto) Se tendían en formato calendario 1 al 15 y del 16 al 30/31. Estas limitaciones varían de usuario a usuario y de nivel de responsabilidad en cabina.

El incumplimiento de estas reglas puede variar de usuario a usuario por estas razones: dependencia de la precisión de los datos, limitaciones técnicas, cobertura geográfica limitada, falta de información detallada, Posibles falsas alarmas, dependencia de la comunicación y cooperación de las aerolíneas y limitaciones de tiempo.

La variación por nivel de responsabilidad en cabina de estas limitaciones incluye la experiencia y calificaciones, procedimientos operativos estándar, restricciones de tiempo de vuelo y descanso, prohibiciones de consumo de alcohol y drogas, limitaciones de salud y autoridad del capitán.

Los problemas en estos casos mencionados suceden frente al piloto y la aerolínea ya que, pueden incurrir en vulnerar la ley estipulada y brindar datos incorrectos sobre los vuelos de sus pilotos y de la experiencia de vuelo de estos. Algunos de los problemas que se han evidenciado son vulnerar reglamentación local y sobre procesos frente a los usuarios y auditores aéreos.

Como pregunta de investigación: ¿Cuál es el impacto que se puede lograr en el proceso de generación de la bitácora de vuelo y la gestión del seguimiento de las horas de vuelo mensuales, trimestrales, semestrales y anuales de los pilotos de la Aerolínea Latam en Colombia mediante la utilización de una plataforma de software web y móvil híbrida?

3.JUSTIFICACIÓN

Se necesita que el plan para una supervisión continua de alta calidad sobre las horas de vuelo de los pilotos sea tan consistente como sea necesario y en todo momento esté alineado con los estándares de seguridad y rendimiento que se esperaría un profesional de la aviación. Desde una perspectiva humana, esta propuesta es para evitar que los pilotos desarrollen fatiga operativa o se encuentren en altos niveles de estrés, el desgaste físico y las posibles quejas musculares. Cuando se puede llevar un seguimiento confiable de las horas de vuelo y los períodos operativos, los pilotos están tomando decisiones informadas que aseguran que no están bajo ningún riesgo al momento de operar en sus respectivas flotas.

Culturalmente, la idea es que el piloto pueda adoptar hábitos de manera más efectiva, eficiente y responsable, para estructurar y administrar su información respecto al tiempo de vuelo. Además de inculcar la cultura del cuidado por el bien común, generará un sentido más amplio en el comportamiento en toda la industria de la aviación. Esto permite que otros pilotos lo repliquen y demuestra que es factible registrar las horas de vuelo de una manera mucho más fácil para los pilotos.

Así el trabajo se centra en la eficiencia y el valor añadido para el piloto, su núcleo es la optimización del tiempo, la organización de la información y evitar esfuerzos innecesarios en la gestión de las horas de vuelo. La plataforma dará más apoyo en procesos que normalmente dependen de esfuerzos humanos y seguimiento manual al ofrecer un sistema de aplicación, robusto y fácil de seguir.

La aplicación estará disponible para un piloto con cualquier dispositivo ya que inicialmente la operación se cubrirá con aportes que realizarán los establecimientos registrados como retribución a los beneficios que esperan recibir respecto al uso de la aplicación. En cuanto a lo económico, los costos de operación mensual para cubrir aspectos de infraestructura serían alrededor de 40\$,por lo que inicialmente el objetivo es lograr recaudar ese valor mínimo. Su modelo multiplataforma y el acceso continuo permitirán a los pilotos mantener registros y consultarlos en cualquier momento con acceso a internet. Además, estará posicionada como un socio en el que se puede confiar, ya sea por pilotos comerciales profesionales o privados y es conveniente para registrar, consultar o presentar información de vuelo. También ayuda a garantizar que las actividades aeronáuticas estén adecuadamente supervisadas al proporcionar información estructurada, confiable y fácilmente accesible cuando se necesita verificar vuelos. Por ejemplo, se ofrecen herramientas para gestionar los registros de vuelo como las horas mensuales, trimestrales, semestrales y anuales. Esto permite a los pilotos mantener consistentemente el foco de sus actividades aéreas, optimizar su planificación en el campo táctico y cumplir con sus obligaciones profesionales.

4. OBJETIVOS

4.1 Generales

Implementar una Plataforma de software web y móvil híbrida para generar bitácora de vuelo y hacer seguimiento de las horas de vuelo mensuales, trimestrales, semestrales y anuales de los pilotos de la Aerolínea Latam en Colombia.

4.2 Específicos

1. Planificar los procesos de ingeniería de requisitos para la plataforma de software web y móvil híbrida.
2. Diseñar estructuras, arquitecturas de software y bases de datos para la plataforma de software web y móvil híbrida.
3. Desarrollar la plataforma de software web y móvil híbrida.

5. MARCO TEÓRICO

El marco teórico de este proyecto se centra en tres temáticas fundamentales: la aeronáutica, específicamente la gestión de vuelos y la importancia de la bitácora de vuelo; la parte de prevención de fatiga en procedimientos del piloto y el número de aterrizajes considerando estas para las aplicaciones tecnológicas que ya vimos en los antecedentes en el ámbito aeronáutico.

5.1 Prevenciones de fatiga

En todo proceso en el que el piloto realice vuelos mediante cualquiera de los roles PM,PFL,PF y PFTO está propenso a sufrir enfermedades como estrés, desorientación espacial y fatiga operativa, síntomas como tener el cuerpo adolorido en lugar de un dolor de cabeza y tener ojos nublados la cual conduce a la disminución en la habilidad para llevar a cabo tareas que requieren destreza, concentración y procesos de alto nivel intelectual (UAEAC, 2023).

5.2 Número de aterrizajes

Todo piloto que pertenezca a una aerolínea y realice vuelos ya sea nacionales o internacionales está en la posición de cumplir la regla de realizar mínimo 3 aterrizajes en 90 días, también conocida como el requisito de moneda(UAEAC, 2023), si no se lleva a cabo, el piloto enfrentará graves consecuencias económicas indirectas, puede perder la habilitación para volar con pasajeros, lo que conlleva la suspensión de sueldo, al no poder volar, la aerolínea puede suspender su salario así mismo tampoco se le podrán hacer asignaciones de vuelo, deberá pagar horas de vuelo con instructor o según la normativa para recuperar la moneda, también esto puede llevar a una posible pérdida de empleo, para poder volar de nuevo deberá realizar los aterrizajes necesarios bajo supervisión o en simulador, incurriendo en gastos de operación.

5.3 Aeronáutica - Bitácora de Vuelo

Definición técnica:

La bitácora de vuelo es un documento esencial para cualquier piloto(OACI, 2006), que va más allá de hacer un simple registro de horas de vuelo. Este documento consigna información crucial sobre el vuelo realizado, el estado y rendimiento del piloto, actividades de vuelo, horas de operación de la aeronave, distintas obligaciones y aproximaciones o requerimientos basadas por ley. Además, se destaca su papel en el cumplimiento de regulaciones de aviación civil para garantizar la seguridad de la tripulación y pasajeros.

Cálculos de horas de vuelo

Todo piloto en su bitácora diaria deberá realizar el proceso de escribir y calcular las horas de vuelo siguiendo el orden OUT,OFF,ON,IN teniendo en cuenta que este proceso es el que les permite a ellos cumplir frente a sus autoridades normativas, mejorar y entender el procedimiento de cada experiencia de los vuelos desde el conocimiento que poseen para asegurar el mantenimiento de la aeronave y demostrar experiencia acumulada(UAEAC, 2023)

Tiempo de vuelo:

El tiempo de vuelo no es solamente un componente que impacta en aspectos económicos como el costo del viaje y en la logística del aeropuerto, si no que también sirve para calcular la duración de un viaje ya que es un resultado que puede medirse en horas desde el inicio del desplazamiento de la aeronave por la pista de despegue hasta el estacionamiento de esta en el aeropuerto en que aterriza o desde el encendido hasta el apagado de los motores.

Cómo se llena una bitácora de vuelo:

El proceso para registrar una bitácora de vuelo exige atención y precisión ya que debe incluir información sobre itinerario, eventos durante el vuelo, horas de vuelo y descanso, firmas del piloto y copiloto. Registrar toda la información correctamente cumple normas de seguridad aérea, salvaguardando la integridad de los registros de vuelo.

Elementos de una bitácora de vuelo:

Los elementos que permiten registrar una bitácora de vuelo son el folio, el cual es el número consecutivo, único y secuencial asignado a cada página, modelo y matrícula de la aeronave, tipo de vuelo, tiempo en cabina, número de aterrizajes, ciclos

de motor, lecturas de instrumentos y más. Al registrar cada uno de ellos se contribuye a la recopilación detallada de datos para análisis posteriores.

6.DISEÑO METODOLÓGICO

6.1 Fase 1: Diseño estratégico — El Qué

En esta fase se hicieron los primeros descubrimientos del problema a trabajar, pues lo ideal es identificar que es lo que realmente importa, lo que más le genera valor al proyecto.

6.1.2 Visión: ¿Hasta dónde vamos a llegar?

Lo primero que se hizo fue describir la visión del proyecto. Esto sonará sencillo, pero en la práctica es donde se traza la línea entre lo que vamos a construir y lo que no. Cuando no hay un plan o una idea clara, cualquier nueva característica parece lo suficientemente buena y el alcance está completamente abierto.

En cuanto a FlightHours nos damos cuenta de que va dirigido a pilotos que deben tener en cuenta el proceso de llenar sus horas de vuelo en sus bitácoras, cada uno compartiría en el sistema el rol de piloto y en la bitácora se usaría como primer oficial y capitán, nos damos cuenta de qué queremos resolver, cuál es el impacto que queremos dar, qué problemas queremos solucionar y dar seguimiento de sus horas generando alertas para evitar problemas con sus respectivas autoridades. Para más información sobre la visión ver tabla 2, anexo 1.

6.1.3 Mapa de impacto

Habiendo identificado el alcance en el que se quería trabajar, se comenzó con el mapa de impacto. Esta estrategia permitió mapear los objetivos del proyecto en los pasos tangibles que se debe tomar para abordar las preguntas: ¿Por qué lo estamos haciendo?, ¿Quién se ve afectado?, ¿Cómo haremos el impacto deseado? y ¿Qué necesitamos construir para lograrlo?

El Mapa de Impacto es un instrumento de planificación estratégica desarrollado por Gojko Adzic(2012) que vincula cuatro niveles de decisiones de planificación. El primero es el objetivo del negocio, derivado directamente de la visión. El segundo son los actores a quienes pretendemos influir, es decir, las personas cuyo comportamiento dentro del proceso queremos cambiar, en mi caso son los pilotos, para más información sobre el mapa de impacto ver tabla 2, anexo 2 .

6.1.4 Modelo de Dominio

Con el modelo de dominio anémico(Fowler,2003) y el enfoque de Domain-Driven Design(Evans,2003) se profundizó en mirar qué objetos hay en el problema y cuáles son sus características y qué conexiones tienen ellos con sus respectivas funcionalidades, en este caso vemos que aeropuerto tiene que ver con tipo de aeropuerto, ruta, ruta aerolínea y aerolínea, fabricante con familia y tipo de la aeronave junto con el modelo, el motor y todo lo relacionado con la bitácora diaria y toda su información, para más información sobre el mapa de impacto ver tabla 2, anexo 3.

6.1.4.1 Modelo enriquecido

De acuerdo con el esqueleto que se mencionó en el modelo anémico, lo que sucede en este modelo(Evans,2003) es tomar cada objeto de dominio con su formato y tipo de datos,decidir si es obligatorio o no, mencionar si se autogenera y si permite valores en blanco etc. También se asignaron sus responsabilidades, es decir, las operaciones que el objeto puede realizar dentro del negocio por ejemplo, en el objeto bitácora diaria, sus responsabilidades incluyen consultar, editar y eliminar la información,cada una con sus entradas, salidas y políticas de negocio asociadas. Estas políticas codifican reglas de validación específicas: tipo de datos, longitud, estado obligatorio, formato y rango, para más información ver tabla 2, anexo 3.

6.1.5 Event storming

Este método propuesto por Brandolini (2021), consiste en identificar todos los eventos que ocurren en el negocio y ordenarlos cronológicamente. Un evento se refiere a algo que ya ha sucedido ejemplo: "Bitácora modificada"

La fortaleza del Event Storming es que se debe considerar el verdadero camino del negocio en lugar de tener que pensar en tu cabeza sobre pantallas o tablas de bases de datos. Si mapeamos el evento ``*Aeropuerto activado*`` también nos preguntamos: ¿quién lo está causando? ¿Qué información se necesita? ¿Qué sucede después? ó ¿Puede fallar?. De esas preguntas surgieron comandos, agregados, políticas y modelos de lectura que no había pensado de otra manera.

Por ejemplo, se encontró que el estado de un aeropuerto no era simplemente "activado". Había estados como "desactivado" que tenían reglas particulares: no activas o desactivas un aeropuerto, solo el administrador puede. Estas reglas no eran evidentes cuando dibujé el modelo anémico, pero fueron reconocidas durante el Event Storming, para más información sobre el mapa de impacto ver tabla 2, anexo 4.

6.1.6 Dimensionamiento del producto

Ya teniendo un mapa completo de lo que se necesitaba construir, con el modelo de dominio enriquecido, faltaba algo en que pensar: *¿cuán grande es esto?*

Este proceso es el juego de dimensionar módulos y funcionalidades. Se pensó en esto para evaluar la complejidad de cada parte basada en factores como el número de entidades, reglas de negocio a implementar, integraciones externas (por ejemplo, keycloak) y nivel de interacción del usuario. No todos los módulos tenían el mismo peso de evaluación. Pero registrar a una persona es mucho más fácil que todo el proceso de registrar horas de vuelo en una bitácora, que es ejecutado por un actor, transiciones entre estados y validaciones cruzadas. El dimensionamiento generó una idea de cuán realista sería el esfuerzo e informó la priorización de lo que vendría después.

6.1.7 Mapa de historias de usuario

En esta etapa fue donde se definieron los elementos a construir y las dimensiones específicas de cada componente. Posteriormente, se procedió a organizar dicha información. Esto se logró mediante el Mapeo de Historias de Usuario, una metodología propuesta por Patton (2014).

El mapa se estructuró en dos ejes. En el eje horizontal, se dispusieron las áreas funcionales del sistema tales como: Gestión Tipo Aeropuerto, Gestión Ruta, Gestión Matrícula, Gestión Motor, entre otras.

En el eje vertical que se encuentra en la columna izquierda, se consignaron las versiones, de modo que cada fila representara una iteración del producto. Las historias de usuario se distribuyeron dentro de esta matriz de acuerdo con su área funcional de pertenencia y la versión planificada para su entrega. Las historias situadas en las filas superiores corresponden a las primeras versiones y poseen la máxima prioridad, mientras que aquellas en las filas inferiores fueron programadas para entregas posteriores, para más información sobre el mapa de historias de usuario ver tabla 2, anexo 5.

6.1.8 Product backlog

A partir del mapa de historias de usuario, se creó el Product Backlog (Schwaber y Sutherland, 2020), la lista de historias que componen el producto. A cada historia se le asignó un código desde la primera hasta la última y se clasificaron según su módulo funcional. Para priorizar, se evaluó cada una de ellas basándose en que es lo que genera el valor para el usuario, en ese orden lo primero que se aborda es lo que tiene más sentido, en este caso hacer el registro y el cálculo de las horas de vuelo en la bitácora del piloto. El backlog no fue un evento que se generó de la noche a la mañana, se revisó y a medida que se iba avanzando, se obtenía más información sobre el dominio. Historias que parecían complejas se dividieron en varias y otras que parecían sencillas resultaron ser más difíciles de lo esperado ya que requerían más tiempo e ideas.

6.1.9 Escritura de historias de usuario.

Una vez que se realizó el product backlog, se procedió a hacer el proceso de escribir cuidadosamente una historia de usuario para cada caso. Se trabajó en el formato establecido por el asesor, un formato estandarizado que luego se utilizó como referencia para todo el proceso de desarrollo. Utilizando la estructura Gherkin —"Dado que... cuando... entonces..."— siguiendo el enfoque de Desarrollo Guiado por Comportamiento o BDD (Wynne y Hellesøy, 2017), junto con los criterios de aceptación. Pero más allá de la estructura, se logró definir algo que se encontró bastante útil al final, el código de los mensajes del sistema. En cuanto a los casos funcionales tanto orientados al éxito como al error, cada uno viene con un código de mensaje asociado (por ejemplo, `'MOD-U-REG_EXI_00001'` para decir que el usuario se ha registrado exitosamente). Se refieren directamente a las respuestas HTTP y mensajes que residen en la base de datos. La implementación de esa idea no se pensó desde el primer día, sino que se convirtió en una necesidad cuando se encontró que el mensaje de error estaba por todas partes y no era similar. Una vez hecho, la trazabilidad desde el requisito funcional hasta la respuesta que el usuario podría ver en pantalla estaba completamente formada.

6.1.10 Plan de liberación

A través de un flujo de entrega definido, el plan de liberación se tradujo en acciones para la planificación estratégica, entregando así valor al usuario final; para ello, los hitos de entrega que agrupan conjuntos funcionales de historias de usuario son la base de esta estrategia y facilitan tanto el desarrollo como las pruebas y el incremento progresivo del sistema. Para decidir qué historias van en cada hito, se siguen tres criterios de agrupación:

1. Cada entrega, para que tenga sentido debe representar un incremento tangible y valioso del producto. Los stakeholders o los usuarios son quienes validan ese valor; eso es lo que llamamos generación de valor demostrable.
2. Las historias incluidas dentro de un mismo hito deben guardar una relación lógica entre sí. Mezclar funcionalidades que no tienen que ver unas con otras es algo que se evita siempre. A esto nos referimos con coherencia funcional.
3. Cuando hay funcionalidades con interdependencias técnicas entre ellas, se deben planificar dentro del mismo hito o bien incluirse en uno anterior. Esto garantiza que el desarrollo sea viable y es lo que entendemos por resolución de dependencias técnicas.

Un cronograma para medir el progreso no es lo único que establece la hoja resultante; también funciona como un mecanismo de gestión y de retroalimentación que resulta esencial. Hay puntos de control donde el avance se evalúa, la alineación con los objetivos de cada release se mantiene y si cambian las condiciones del proyecto, se pueden ajustar las prioridades.

7.FASE 2.ARTEFACTOS TÉCNICOS

7.1 Diseño arquitectónico

Tomando las historias de usuario que se hicieron, surgió la idea de usar aquellos elementos fundamentales, ya sean funcionales o técnicos que tienen una influencia crítica en el desarrollo del proyecto y que sirven dictando las decisiones clave sobre su forma, estructura y funcionalidad con eso en mente apareció la tarea de buscar y validar las tecnologías que cumplieran como candidatas, se compararon alternativas de solución y se eligió la que mejor se adaptó a la solución. A partir de esa decisión, se creó el arquetipo de solución que convierte la estrategia en los componentes reales y finalmente se propuso el stack tecnológico para que todo lo anterior se viera reflejado en las herramientas tecnológicas específicas.

7.1.1 Atributos de calidad

Para definir qué tan bueno, seguro y confiable debía ser el sistema se hizo un ejercicio llamado taller de atributos de calidad basado en la metodología propuesta por Bass et al (2021). La idea consistió en evaluar distintas características que una aplicación puede tener, por ejemplo, que sea rápida, que sea segura, que funcione siempre, etc. y decidir cuáles eran las más importantes para el proyecto. Para eso se trabajó con los dos roles del mapa de impacto: el administrador, que es la persona encargada de gestionar aerolíneas, aeropuertos y recursos del sistema y el piloto, que es el usuario principal que registra sus horas de vuelo y consulta su bitácora diaria. A cada rol se le asignó un voto sobre 120 de 15 atributos de calidad, por las características que consideraba más relevantes según sus necesidades.

Los 10 atributos seleccionados y las decisiones tecnológicas asociadas a cada uno fueron disponibilidad, confiabilidad, seguridad, rendimiento, experiencia de usuario, accesibilidad, conformidad, interoperabilidad, escalabilidad y capacidad de ser administrado para más información sobre los atributos de calidad ver tabla 3, anexo 9.

7.1.1.2 Escenarios de calidad

Se crearon escenarios de calidad basados en atributos utilizando la estructura SEI (Bass, Clements & Kazman, 2021). Esto incluyó la definición de 100 escenarios, 55 básicos y 45 alternativos repartidos entre los 10 atributos que se seleccionaron, orientados a que puedan ser probados por el usuario final. Cada escenario describe una situación específica bajo la cual un sistema necesita responder de una manera u otra, los escenarios básicos representan el camino normal donde todo funciona correctamente por ejemplo, el piloto inicia sesión con su contraseña correcta y entra al sistema sin problema; y los escenarios alternativos, son los que simulan situaciones problemáticas o inesperadas para verificar que la aplicación reacciona bien por ejemplo, qué pasa si el piloto escribe una contraseña equivocada o si se cae la conexión a internet, tienen los siguientes elementos: fuente del estímulo, estímulo, entorno, artefacto, respuesta y medida de respuesta. Son estos escenarios los que más tarde nos dan verificación y nos permiten probar que las tácticas y tecnologías seleccionadas realmente cumplen con lo que valoramos respecto a los atributos de calidad que tomamos en consideración, para más información sobre los escenarios de calidad, ver anexo 9.

7.1.1.3 Funcionalidades críticas

Las funcionalidades críticas que aparecen se derivan de las historias de usuario, se agrupan en funcionalidades compuestas y se evalúan con base a dimensiones de impacto económico, social, imagen de la entidad a la que representa y humana. En estas, lo que se busca es asegurar el funcionamiento mínimo reduciendo vulnerabilidades que serían perjudiciales para la columna vertebral de la aplicación, lo que podría crear efectos donde múltiples impactos podrían afectar a individuos, por ejemplo, caídas frecuentes de una aplicación en un campo financiero puede provocar un daño de mala percepción de la imagen de la empresa por parte de sus clientes, en un contexto humano, si la bitácora no hace bien el cálculo de los datos y límites de vuelo a respetar podría causarle al piloto, problemas de salud como estrés o desorientación social, para más información sobre las funcionalidades críticas ver tabla 3, anexo 10.

7.1.1.4 Restricciones técnicas.

Debido a que no hubo un cliente que recomendara cuales usar, para llevar a cabo el proyecto se realizaron 22 buenas prácticas, las cuales estaban organizadas en 5 categorías según el área que cubre cada una, en arquitectura se usó una limpia de 4 capas, en cuanto a API REST se usó un HATEOAS nivel 3 de madurez según el modelo de Richardson(Fowler, 2010), se manejó documentación Swagger/OpenAPI, en la dirección de los servicios se incluyó el v1 lo que permitió hacer cambios futuros sin romper las aplicaciones que ya estaban funcionando en la versión actual, se usó nomenclatura REST en inglés y en plural, en cuanto a seguridad se implementó autenticación JWT con Keycloak y Autorización por roles(RBAC) la cual se definió directamente en las rutas del sistema lo que significaba que el acceso a las funciones se controló según el rol del usuario, se implementó sanitización de entradas, validación con esquemas JSON y todas las restricciones en cuanto a código limpio, mensajes centralizados, centralización de registros técnicos, constantes de dominio tipadas, patrón modular de repositorios, mapeo semántico de errores, filosofía de errores 4XX y persistencia, sentencias SQL preparadas al arranque y transacciones gestionadas en la capa de orquestación, para más información sobre las restricciones tecnicas ver tabla 3, anexo 11.

7.1.1.5 Restricciones de negocio.

En las restricciones de negocio, las cuales son limitaciones del mundo real que influyen en cómo se construyó y hasta donde llega la aplicación, se tuvo la idea de evaluar los factores de riesgo, aquellas condiciones extremas que pudieran comprometer el curso del proyecto dejándolo con funcionalidades incompletas; en el proyecto, se lograron identificar 6 restricciones organizadas en 3 categorías, financiero, operativo y alcance, para más información sobre las restricciones de negocio ver tabla 3, anexo 12.

7.1.2 Spikes

Cuando se entró al tema de los spikes o pruebas de concepto, los cuales son investigaciones técnicas que se hicieron antes de tomar decisiones importantes sobre qué herramientas y tecnologías usar en el proyecto, se encontraron varias opciones posibles, se hizo una investigación comparando alternativas y probando cual funcionaba mejor para las necesidades, se realizaron 14 spikes en los que por ejemplo si pensaba en “¿cómo resolver la duda técnica” o “¿Cómo enviar correos de verificación?”, se haría un spike para validarlo con una prueba de concepto, con esa idea clara y resuelta, se usó la estructura indicada en la que se verificaron diferentes herramientas y tecnologías, revisando varios aspectos como la parte económica, la accesibilidad y la integración entre las que ya se iban seleccionando, para más información sobre los spikes ver tabla 3, anexo 13.

7.1.3 Alternativas de solución.

En esta fase, la alternativa de solución que se escogió mediante el estudio previo de tácticas y estrategias, lo cual se hizo en los drivers arquitectónicos y los spikes, los cuales ayudaron a hacer la evaluación de las tres alternativas que se plantearon. Se terminó escogiendo la alternativa 3, debido a que era la que cumplía con los atributos de mayor prioridad para el usuario final y que a su vez, coincidía con los spikes que se adoptaron, para más información sobre las alternativas de solución ver tabla 3, anexo 14.

7.1.4 Arquetipo de solución.

Dado lo anterior, cuando se habló de los elementos estratégicos y técnicos, en ese momento lo que era derivar el arquetipo de solución tanto del Frontend como del Backend era factible. Esto empezó a ser la guía que funciona como una traducción técnica de la necesidad del problema, ya que en la plantilla asignada se escribió el inventario completo de todas las herramientas, tecnologías y servicios que componen la aplicación, se hizo mencionando cual se usó, quien la fabrica, si es gratuita o paga y por qué se eligió, para más información sobre el arquetipo de solución ver tabla 3, anexo 15.

7.1.5 Plataforma tecnológica.

Como se mencionó anteriormente en el arquetipo, lo que se hizo fue enlistar toda la información necesaria sobre las herramientas que se usaron en el proyecto, cuando se llegó a la Plataforma tecnológica, la cual es el mapa completo de todas las tecnologías que se usaron en el proyecto, se presentaron resumida y organizadamente con el fin de dar una vista panorámica con las columnas Categoría y Tecnología para mostrar cuales se usaron en cada parte del sistema, de los cuales en total son 53 elementos distribuidos en 6 secciones, para más información sobre la plataforma tecnológica ver tabla 3, anexo 16.

7.2 Diseño detallado.

En esta etapa se muestra cómo se ve la solución del problema funcionando por dentro. Los diagramas muestran la estructura interna del sistema, a través de ellos se ve cómo se conectan las estructuras, cómo fluyen los datos, el modelo de datos relacional y cómo se hizo el despliegue.

7.2.1 Diagrama de structs.

El diagrama de clases, que en Go se traduce como un diagrama de estructuras, es el mapa de toda la información que domina el sistema. Muestra todas las estructuras según el modelo de dominio (Evans, 2003) que existen dentro de la aplicación, pilotos, aerolíneas, aeropuertos, vuelos, etc. qué datos tiene cada una y como se relacionan entre sí, las structs en Go son como una ficha con campos de información, se tienen 25 entidades, 19 structs y 6 enums que son listas de valores fijos.

En go no se utiliza herencia, sino composición mediante structs embebidos e interfaces, esto último se refleja en el diagrama como relaciones de composición en lugar de jerarquías de herencia.

Las entidades principales del dominio son Employee(Empleado), Airline(Aerolínea), AirlineEmployee(EmpleadoAerolínea), Airport(Aeropuerto), Manufacturer(Fabricante), Engine(Motor), AircraftFamily(Familia de Aeronaves), AircraftModel(Modelo de Aeronave), TailNumber(Matricula), Route(Ruta), AirlineRoute(Ruta de Aerolínea), Daily Logbook(Bitácora Diaria), DailyLogbookDetail(Detalle de Bitácora Diaria), EmployeeFlightSummary(Resumen de Vuelo del Empleado), FlightHoursSummary(Resumen de Horas de Vuelo), Message(Mensajes), PilotRoleBreakdown(Desglose por Rol de Piloto), FlightAlert(Alerta de Vuelo) y PeriodInfo(Información de periodo). Cada una se modela como un struct en Go y cuando una entidad necesita datos de otra se usa composición, para más información sobre el diagrama de clases ver tabla 3, anexo 17.

7.2.2 Diagrama de componentes.

En este diagrama lo que se hizo fue mostrar las piezas(componentes) que forman la aplicación y cómo se conectan entre sí ya que cada una es una parte del sistema que se encarga de una tarea específica.

Los componentes identificados a partir de las funcionalidades del sistema, separando cada módulo de negocio como una unidad independiente. En el frontend, la zona donde se muestra todo lo que el usuario puede ver y tocar, por ejemplo, en la primera fila están los módulos de autenticación, login,register, reset password , email verification, los cuales son la entrada a la aplicación. En la segunda fila están las vistas principales, perfil del usuario, Bitácora, Detalle Bitácora y el Dashboard del administrador. En la tercera fila está el campo operacional que conlleva Vuelos, Resumen Vuelos, Alertas y Matrículas. En la cuarta fila están los datos maestros, aerolíneas, aeropuertos, rutas y rutas aerolínea. En la quinta fila están los catálogos técnicos, modelo de la aeronave, fabricantes, tipo tripulante y splash. Adicionalmente, hay una barra gris que representa las herramientas internas que todos los componentes usan por detrás como Dio, Kiwi, Bloc y Validators.

En el backend nos encontramos con las piezas que el usuario no ve, es donde sucede toda la magia, se guardan los datos, se hacen cálculos y se aplican las reglas del negocio. En la primera fila están los componentes employee, que gestiona todo lo relacionado con empleados, pilotos y personal, aerolínea, aeropuertos y modelo de la aeronave. En la segunda fila encontramos ruta, ruta aerolínea, matrícula y motores. En la tercera fila están bitácora diaria, detalle bitácora, resumen vuelos y fabricantes. En la cuarta fila están los componentes, mensajes del sistema y tipo tripulante.

En la parte inferior están dos barras, una barra café que es donde se muestran las herramientas internas del back, las cuales son Gin/Echo, MySQL Driver, JWT validator y el JSON Schema y la segunda que es la barra Crosscutting que es la barra dorada, la cual representa funciones que cruzan todos los componentes, Auth(Autenticación/Autorización) y Monitoreo.

En la esquina inferior derecha están los servicios externos, Keycloak, MySQL, Prometheus, Grafana+Loki, Swagger, Docker, Dokploy, SonarQube y Traefik para más información sobre el diagrama de componentes ver tabla 3, anexo 18.

7.2.3 Diagrama de paquetes.

Cuando se llegó a la creación de este diagrama se realizó este plan, en vez de poner todo en un solo lugar, se organizó todo en secciones, esto es lo que se llama paquetes, una sección o carpeta que agrupa código relacionado. Con esa idea en mente se dibujó el diagrama que muestra cómo todo está organizado internamente en el proyecto. Aquí se muestran 3 paquetes, los de Backend, que son 52 con 29 conexiones, los de Frontend que son 35 con 44 conexiones y los de FullStack “HTTP/REST(Dio a Gin Router)”, para más información sobre el diagrama de paquetes ver tabla 3, anexo 19.

7.2.4 Diagrama de secuencia.

Después de dibujar el diagrama de componentes, en el que se mostraba que piezas tenía el sistema junto con el diagrama de paquetes, en el cual se dijo cómo están organizados las carpetas, se realizó la idea de que en este diagrama se podría dibujar qué pasa cuando un usuario hace algo, haciendo referencia a las funcionalidades como registrar detalle de bitácora de vuelo, la cual es la HU 14, mostrar el proceso de esa operación como si fuera una historia y como esta toca y pasa por todas las capas para más información sobre el diagrama de secuencia, ver anexo 20.

7.2.5 Diagrama de despliegue.

Cuando se realizó el despliegue con todo el sistema funcionando, salió esta pregunta, ¿cómo dibujo todo este procedimiento? y se hizo esta comparación, tenemos una app que es como una casa, necesitamos saber donde va a vivir esa casa, que terreno va a tener, que servicios va a tener etc, se eligió la idea de mostrar eso exactamente en este diagrama, se decidió mostrar donde se instala cada parte de la aplicación y cómo se conectan entre sí y cómo llega a los usuarios.

Funcionó así, tenemos la zona 1, la cual es el servidor VPS, todo el sistema FlightHours vive en un solo servidor alquilado en la nube, que tiene como sistema operativo Ubuntu 24.04 LTS, 6 hilos virtuales y una dirección IP, dentro de este hay 3 grupos de servidores, grupo 1 Dokploy automático, el cual es como un robot que sube la aplicación automáticamente cada vez que hay cambios en el código; dentro de dokploy viven 3 cosas, la primera es Traefik, que es un proxy inverso, el cual recibe todas las visitas que llegan al servidor y las envía al lugar correcto, la segunda un contenedor, el backend de la api con tecnología Go 1.25 + Gin, el cual procesa toda la lógica junto con el puerto 9081 y la tercera un contenedor frontend web con Flutter 3.32 compilado y el puerto interno 90 que sirve la versión web de la aplicación.

En el grupo 2 Docker Compose, aquí se tienen los servicios con Datos Permanentes, los cuales se gestionan por separado de Dokploy porque guardan datos importantes que no se deben perder estos son, primero esta Keycloak 24, con el protocolo OAuth2/OpenID Connect, en el dominio previamente configurado y permite gestionar usuarios, contraseñas, roles y tokens de acceso; el segundo es MySQL9.0 que es como una bodega de datos en el puerto 3306, que está conectada con la Base de datos flightDb, la cual guarda datos de vuelos y keycloakDb que guarda datos de autenticación, lo que se guarda en la bodega es todos los datos del sistema.

En el grupo 3 Stack de observabilidad, la responsabilidad de este grupo es vigilar que todo funcione correctamente, posee las herramientas Promtail que recolecta logs de api, Loki, que almacena y organiza todos esos logs para que se puedan buscar, Prometheus, que recolecta métricas numéricas, teniendo en cuenta cuantas peticiones y cuanto tardan, y Grafana, que muestra gráficas bonitas con los datos de loki y prometheus, se conectan de la siguiente manera, el backend escribe logs, Promtail recibe y envía, loki consulta y llegan a grafana.

Después de hacer todo lo de la zona 1, se pasó a la zona 2 que consiste en el Dispositivo Móvil, el cual tiene plataformas Android y IOS, tecnología Flutter + Dart y se conecta enviando peticiones HTTPS al servidor a través de Cloudflare, luego vamos a traefik y luego al backend.

En la zona 3, que es el navegador web se conecta accediendo al dominio `flighthours.rbsupport.com` va a cloudflare, luego a traefik y después a frontend Nginx. Cuando llegamos a la zona 4 que son los servicios externos en la nube, estos no viven en el servidor VPS, los cuales son Cloudflare, que protege el sitio de ataques y gestiona los dominios(DNS) y es la primera capa de seguridad antes de llegar al servidor, Resend, que envía correos electrónicos automáticos, verificación de cuenta y recuperación de contraseñas, Let's Encrypt, que emite certificados SSL gratuitos para que las conexiones sean seguras, GitHub, que almacena el código fuente y activa despliegues automáticos cuando hay cambios y SonarCloud, que analiza el código para detectar errores, vulnerabilidades y malas prácticas para más información sobre diagrama de despliegue, ver anexo 21.

7.2.6 Modelo entidad relación.

Durante la creación de este modelo, se estudió la base de datos flightDb, la cual tiene 13 tablas, organizadas en 4 grupos según su función, en cada una de las tablas se ven las entidades y datos de todas, los nombres de las tablas y columnas están en inglés siguiendo el estándar técnico del proyecto, mientras que los datos almacenados están en español.

Se describe así, en el Grupo 1, la tabla employee(empleados) se ve toda la información de un empleado registrado en el sistema; en el grupo 2 que es operación, se tiene en cuenta la tabla airline(Aerolínea), la cual guarda toda la información de una aerolínea registrada, lo mismo sucede con la tabla airport(Aeropuerto), route(Ruta), airline_route(Ruta aerolínea), system_messages(mensajes del sistema) la cual es independiente; en el grupo 3 el cual es la Flota(Aviones) tenemos las tablas manufacturer(Fabricante), engine(Motor), aircraft_model(Modelo de aeronave) y tail_number(Matricula), finalmente en el grupo 4 corresponde a Bitácoras y Vuelos, lo que consiste en las tablas daily_logbook(Bitácora Diaria) y daily_logbook_detail(Detalle Bitácora diaria), para más información sobre el modelo entidad relación, ver anexo 22.

8. FASE 3.CONSTRUCCIÓN

8.1 Análisis estático

Cuando se llegó a este análisis, se tenía todo el código funcionando tanto Front como Back, surgió la necesidad de hacer un análisis de todo el proyecto sin ejecutarlo y mirar qué problemas potenciales salían, con esa decisión tomada, se decidió usar SonarQube y realizar el análisis, este artefacto muestra qué tanta deuda técnica se tenía.

En la hoja de resultados del archivo de excel que se usó, el quality gate dice que tanto el Backend como el Frontend cumplieron las condiciones mínimas de calidad. En cuanto a cobertura en los dos proyectos, Back tiene 94.8% y el Front exactamente el 90.0%, ambos superan el umbral de ser mayor o igual a 90%, lo que significa que si el código tiene 100 líneas y las pruebas ejecutan 94 de esas líneas, la cobertura es 94%, esto significa que ese porcentaje del código, está verificado por pruebas unitarias.

Teniendo en cuenta los datos del proyecto back con 23.000 líneas y en front con 30.300 , cerca de un ratio test/código de 1.59 veces y las duplicaciones están controladas (1.8% backend, 0.0% frontend). Los únicos code smells restantes, que son 9, están en archivos auto-generados de plataforma del frontend, no en el código del proyecto.

En cuanto a las métricas como bugs, Vulnerabilidades, Security Hotspots, code smells, deuda técnica y duplicaciones, el Backend sacó, siguiendo el orden, 0 (Rating A), 0 (Rating A), 0 (100% revisados), 0 (Rating A), 0h y 1.8%; en cuanto al Front, 0 (Rating A), 0 (Rating A), 0 (100% revisados), 9 (Rating A), 1h 2min y 0.0%.

En la hoja Herramientas se mencionó las tecnologías que se usaron, en Backend sonar-scanner, el cual envia el código a SonarCloud para análisis automáticamente con cada push en GitHub, golangci-lint, linter de Go que revisa estilo y errores antes de cada commit(pre-commit hook), go test -cover, genera reporte de cobertura de pruebas en el pipeline de CI/CD; en Front use flutter_lints, linter de Dart que revisa estilo y errores en el IDE y con flutter analyze y flutter test –coverage que genera reporte de cobertura y se realiza manualmente. Se llevó a cabo la configuración de Quality Gate que define condiciones mínimas para aprobar, Quality Profile que define las reglas que se aplican a cada lenguaje, en mi caso Go y Dart y Exclusiones, archivos que no se analizan, código generado automáticamente(g.dart, *.freezed.dart), carpetas de terceros (vendor/), documentación, etc).

En la hoja Hallazgos lo que se encontró, con base en los ratings Hal-001 y Hal-002 en estado medio, con estado alto HAL-003, HAL-004, HAL-005, HAL-006, HAL-007, HAL-008, HAL-009 y HAL-010 en estado positivo, en la hoja que es de configuración están las configuraciones clave, todo coincide con el archivo que use, para más información sobre el análisis estático, ver anexo 23.

8.2 Devops

Durante todo este proceso del proyecto, se usó una plantilla en la que se describe que el pipeline CI/CD de FlightHours automatiza todo el flujo desde que se sube código al servidor hasta que esté disponible para los usuarios, se clasifica en 5 etapas.

En la primera etapa, llamada control de versiones, esta no es automática sino una práctica para mantener el código organizado usando las siguientes herramientas, la primera es Git + Git Flow (Driessen,2010), un método para organizar las ramas del código, se usa main, develop, feature/*,release/* y hotfix/*, cada vez que develop se fusiona con main,se crea un tag de versión(v0.1.0) lo cual es como ponerle un sello de “Version 0.1.0” al producto; la segunda es Conventional Commits(Conventional Commits,s.f), la cual usa como tipos de commit, el tipo feat como nueva funcionalidad,fix como corrección de error, refactor como mejora sin cambiar comportamiento, test cómo agregar o modificar pruebas, docs como documentación y chore como tareas de mantenimiento, lo que sucede es como llevar un diario del proyecto bien organizado, en vez de escribir frases como “cambios varios”, se escribe exactamente que se hizo y por qué.

En la segunda etapa, llamada Integración Continua, CI, se verifica la calidad, cada vez que alguien sube código, las siguientes herramientas se ejecutan automáticamente para asegurarse de que todo funciona, la primera es Go Test + coverage en el backend, la cual ejecuta todas las pruebas unitarias, es como un examen automático, cada vez que se sube el código, el sistema corre todas las pruebas y dice si todo está bien o si algo falló; la segunda es Flutter Test + Icov en el frontend, ésta ejecuta todas las pruebas del frontend unitarias con el comando `flutter test —coverage` con el objetivo de que la cobertura sea mayor o igual al 90%; la tercera es SonarCloud, analiza el código estáticamente buscando bugs, vulnerabilidades y malas prácticas de manera automática después de las pruebas, si Go Test funciona como el examen, esta sirve como el inspector de calidad que revisa no solo que funcione, sino que el código sea limpio, seguro y mantenible.

En la tercera etapa, llamada Construcción, se empaqueta la aplicación, una vez que el código pasó las pruebas, se envuelve en una imagen de Docker lista para desplegar, esto sucede gracias a las siguientes herramientas, la primera es Docker + Dockerfile en el back usando las 2 etapas del Dockerbuild, Build, que compila el código Go en un binario ejecutable y Run que copia solo el binario a una imagen mínima. La segunda es Docker Compose, la cual orquesta múltiples contenedores que trabajan juntos, en desarrollo esta levanta servicios como MySQL, Keycloak, Swagger ui cada uno en su contenedor con el comando `docker-compose up` y genera en producción files separados para Grafana, Prometheus, Loki y Promtail.

En la cuarta etapa, llamada Despliegue Continuo,CD, sucedió el proceso de subir a producción, aquí es donde la aplicación empaquetada pasa a producción automáticamente, sucedió gracias a las siguientes herramientas, la primera fue Github Webhooks, la cual envió una notificación automática a Dokploy cuando hubo cambios en main, funcionó como un timbre automático. La segunda es Dokploy, la cual recibió el aviso, clonó el código, construyó la imagen de docker y desplegó y es capaz de soportar rollbacks automáticos, variables de entorno y volúmenes. La tercera es Traefik que se encargó de detectar los nuevos contenedores y redirigir el tráfico automáticamente, cuando Dokploy desplegó un contenedor nuevo, esta lo detectó al instante mediante labels de docker y empezó a enviarle tráfico. La última que se usó es Cloudflare, esta gestionó DNS, certificados SSL públicos y otorgó protección contra ataques como DDoS y WAF(Web Application Firewall), funcionó como la primera línea de defensa.

En la quinta etapa, llamada Monitoreo Post-Despliegue, es la que se encargó de vigilar que todo funcionara una vez que se desplegará la app, las siguientes herramientas vigilaban continuamente que todo estuviera bien, la primera era Prometheus que recolectaba métricas numéricas del backend cada 15 segundos, media constantemente cuantas peticiones llegan, cuanto tardan y cuantas fallan, la segunda es grafana que mostraba gráficas en tiempo real con los datos de loki y prometheus, si este era el medidor, esta era como la pantalla del centro de control donde ves las gráficas y las alertas; la última que se usa es loki + Promtail, esta capturó los logs de los contenedores de docker automáticamente, ya que loki almacenó y permitió buscar en los logs, promtail recogió todos los registros de actividad y los archivó en un catálogo, cuando algo fallaba, se podía buscar en Grafana y encontrar el error exacto para más información sobre los escenarios de calidad, ver anexo 28.

8.3 Pruebas de carga

Para las pruebas de carga se usó K6, que es la herramienta de Grafana para ese tipo de pruebas. La idea era averiguar cómo se comportaba el sistema del proyecto cuando está bajo presión al realizar varias llamadas a los endpoints, cuantas aguanta y cuantos usuarios pueden usarla y qué problemas saldrían, se creó un escenario y se lanzó hasta 1000 y sacó varios errores en 2 ocasiones cuando la latencia se acercaba a ser mayor que 500, gracias a esos valores se dio cuenta que la api aguantaba 1000 VUs sin caerse y que los principales cuellos de botella del sistema eran los endpoints de alertas, para más información sobre los escenarios de calidad, ver anexo 24.

9.RESULTADOS

9.1 Perspectiva del piloto

El piloto es el usuario principal. Desde su perspectiva él podrá iniciar sesión, registrarse, autenticarse, cambiar la contraseña, modificar la contraseña, actualizar su información, eliminar su sesión, crear, encontrar, editar y eliminar su bitácora de vuelo, ver los registros de las horas de los vuelos que están en la bitácora, ver, editar y guardar cualquier dato en la bitácora de vuelo, consultar las matrículas, ver el modelo y el tipo de la aeronave, las rutas de la aerolínea, alertas relacionadas con el número de aterrizajes mínimos o con el exceso de trabajo, indicar el cargo que ha hecho durante los vuelos y el total de las horas.

9.1.1 Inicio de sesión

Pantalla de inicio de sesión de la aplicación FlightHours. Aquí se presentan los campos para correo electrónico y contraseña, además de la opción de recuperación de contraseña y la registro para nuevos usuarios.

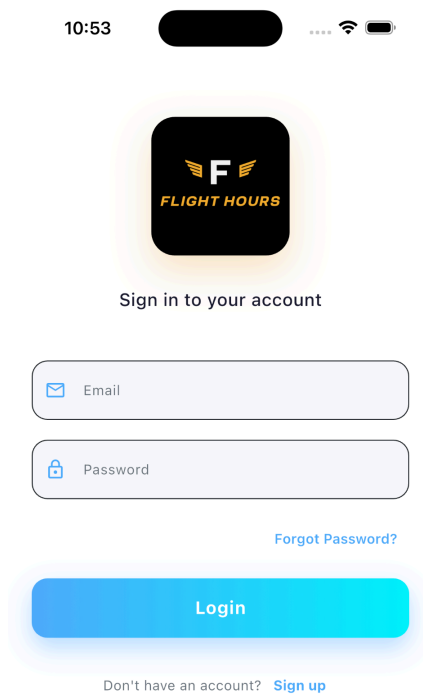


Figura 1: *Inicio de Sesión*

9.1.2 Registro de nuevo piloto

Formulario de registro que solicita información personal del piloto como el número de identificación, nombre, correo electrónico, contraseña y confirmación de contraseña.

The screenshot shows a mobile application interface for a registration form. At the top, the status bar displays the time 11:16, a blacked-out phone number, and icons for cellular signal, Wi-Fi, and battery. The app's header is black with the word 'Register' in white. Below the header, the title 'Personal Information' is centered in bold, followed by the instruction 'Enter your basic details'. The form consists of five input fields, each with a light purple background and a blue icon on the left: 'ID Number' (ID card icon), 'Name' (person icon), 'Email' (envelope icon), 'Password' (lock icon), and 'Confirm Password' (lock icon). The Password and Confirm Password fields have a blue eye icon on the right to toggle visibility. A blue 'Continue' button is positioned below the fields. At the bottom, the text 'Already have an account?' is followed by a blue 'Login' link. A thick black horizontal line is located below the 'Login' link.

11:16 [Redacted] [Signal] [Wi-Fi] [Battery]

Register

Personal Information
Enter your basic details

ID Number

Name

Email

Password

Confirm Password

Continue

Already have an account? [Login](#)

Figura 2 *Registro*

9.1.3 Recuperación de contraseña

Pantalla de recuperación de contraseña que permite al piloto ingresar su correo electrónico para recibir un enlace de restablecimiento.

Figura 3 *Recuperación de contraseña*

9.1.4 Autenticación

Pantalla principal del piloto, que presenta un dashboard con el gráfico circular de distribución operacional. El gráfico muestra un total de las horas acumuladas, distribuidas por rol: PF — Pilot Flying, PFL — Pilot For Landing, PFTO — Pilot For Take-Off y PM — Pilot Monitoring junto con la sección que muestra los registros de horas de vuelo más recientes del piloto, indicando la fecha, y el modelo de aeronave.

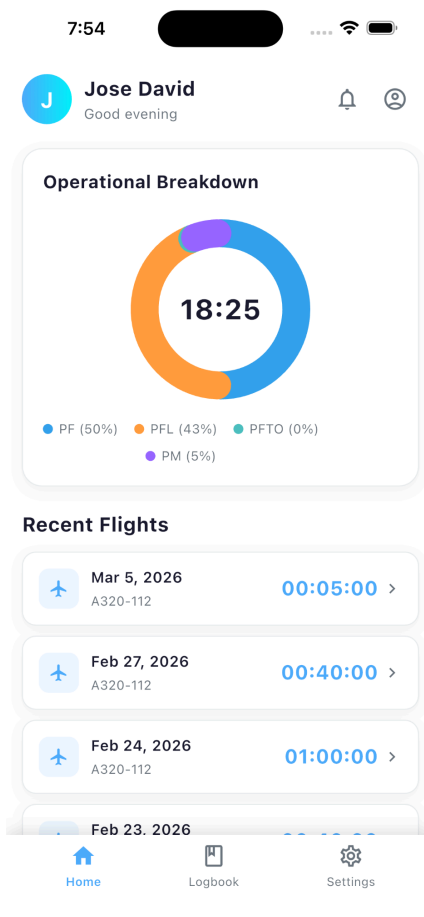


Figura 4 *pantalla de inicio del piloto*

9.1.5 Modificar la contraseña

Pantalla que muestra un formulario para el cambio de contraseña que muestra los requisitos de seguridad, el correo electrónico del usuario y campos para la contraseña actual, nueva contraseña y confirmación.

7:50

Change Password
Update your account security

Password Requirements
Your new password must be at least 8 characters long.

Account Details

Email
josedalogo@yopmail.com

Current Password
Current Password

New Password
New Password
Confirm New Password

Cancel Change Password

Figura 5 *Modificar la contraseña*

9.1.6 Perfil del piloto

Pantalla de perfil del piloto que muestra su rol, estado, número de identificación, número de identidad del piloto dentro de la aerolínea registrada y fechas de vigencia junto con las opciones para editar y eliminar su información.

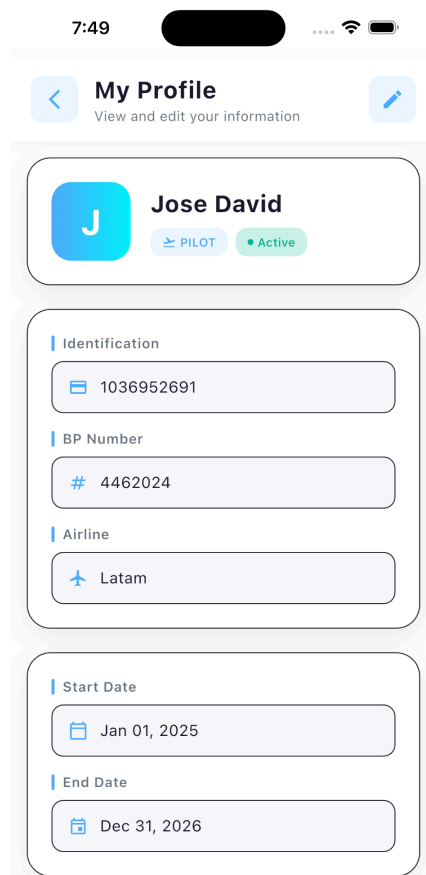


Figura 6 Perfil del piloto

9.1.7 Eliminar información del piloto

Opción que se muestra dentro de la pantalla del perfil del piloto en la que permite la eliminación de la cuenta con una advertencia.

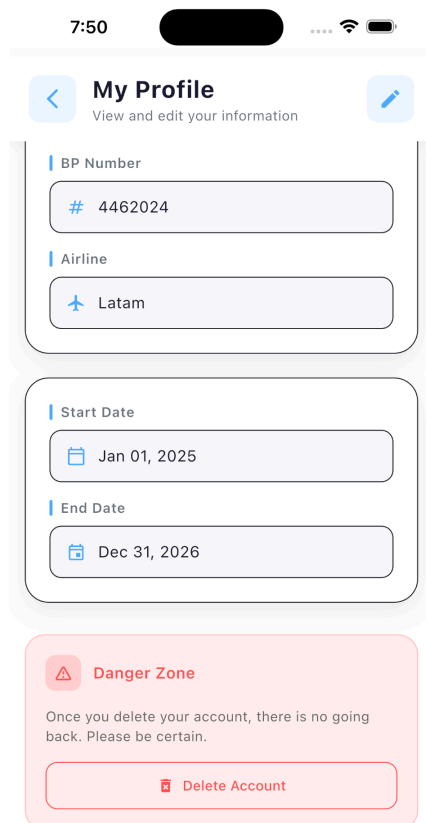


Figura 7 Eliminar información del piloto

9.1.8 Consultar aerolínea

Pantalla de selección de aerolínea que muestra las opciones disponibles Avianca, Latam, Spirit con un buscador por nombre o código, con el fin de que el piloto complete su registro.

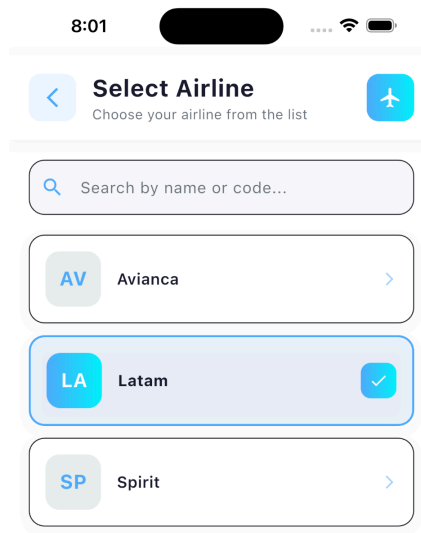


Figura 9. Consultar Aerolínea

9.1.9 Consultar ruta aerolínea

Pantalla que está dentro de la sección de editar datos del vuelo mostrando la lista de las rutas disponibles para su edición.

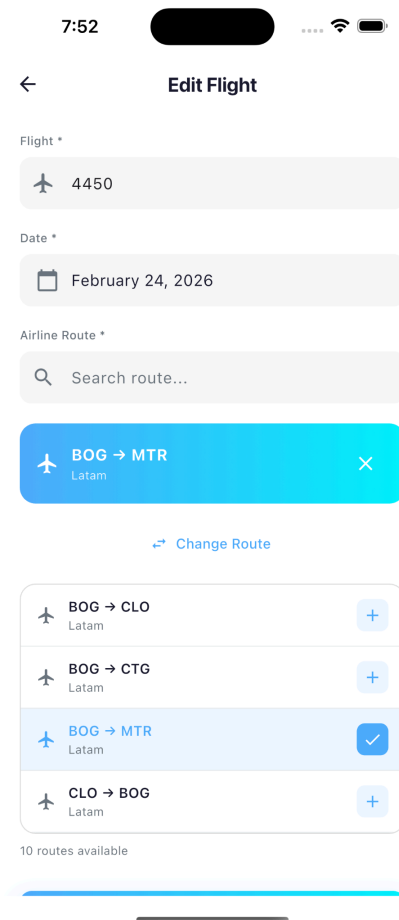


Figura 9. Consultar Ruta Aerolínea

9.1.10 Consultar bitácora diaria

Pantalla principal del módulo de bitácoras, donde el piloto puede ver todas sus bitácoras creadas, ordenadas por fecha. Aquí se muestra la fecha, el número de página del libro y el estado. El piloto puede ver el detalle, activar, desactivar o eliminar una bitácora.

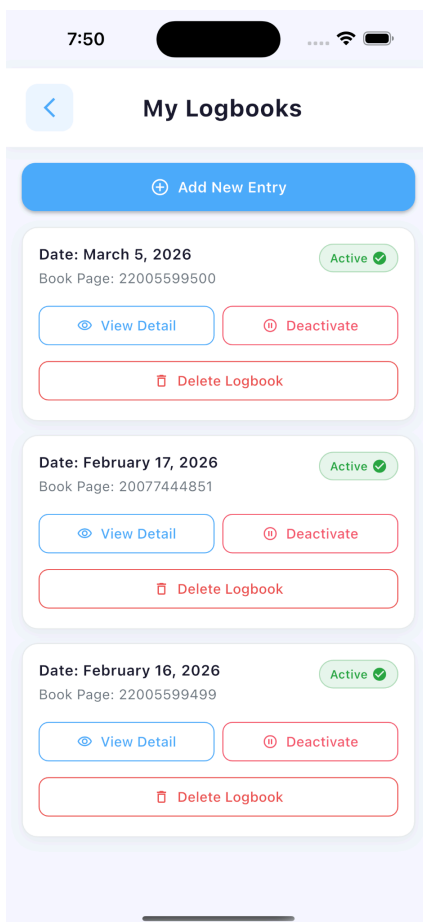


Figura 10 Consultar Bitácora

9.1.11 Agregar bitácora

Menú que permite crear una nueva bitácora, solicita la fecha del registro y opcionalmente el número de página del libro.

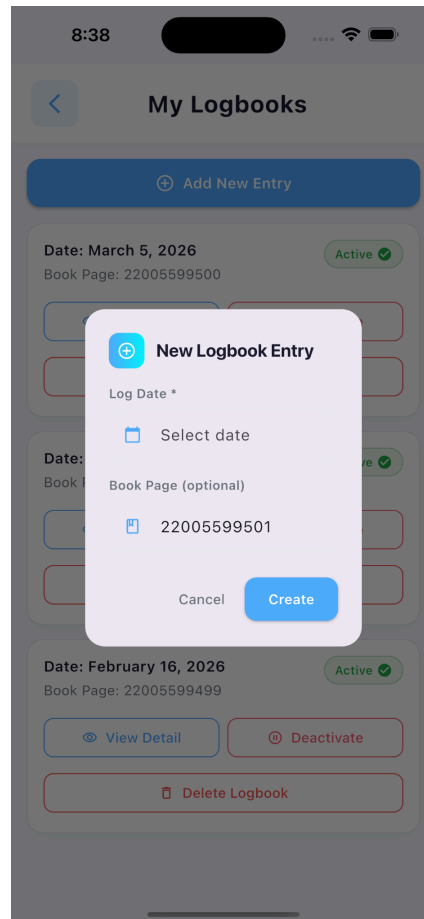


Figura 11. *Agregar Bitácora*

9.1.12 Consultar matrícula

Pantalla de búsqueda de la matrícula de una aeronave con un campo donde el piloto ingresa la matrícula del avión para relacionarla con las horas del vuelo.

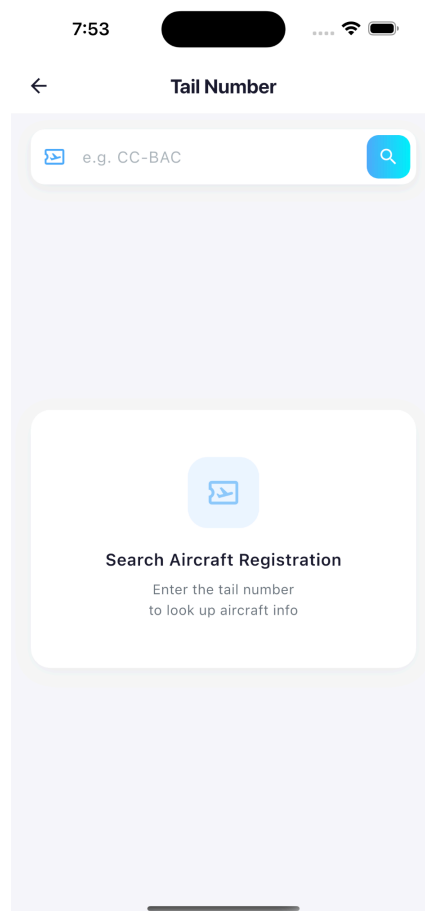


Figura 12. *Consultar Matrícula*

9.1.13 Consultar modelo y tipo de Aeronave

Pantalla que muestra toda la información técnica de la aeronave modelo, aerolínea, tipo de aeronave, motor y estado operacional gracias al número de la matrícula.

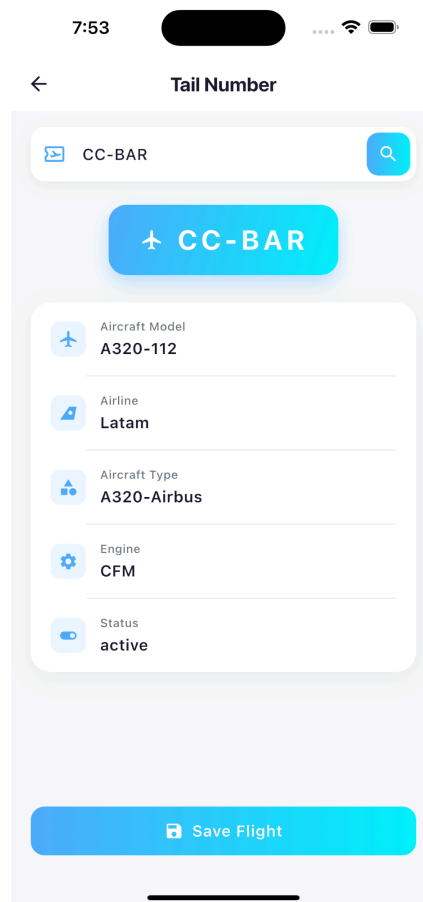


Figura 13. Consultar Modelo y Tipo Aeronave

9.1.14 Consultar los vuelos de una bitácora

Pantalla que muestra los vuelos correspondientes a un día de trabajo y que se guarda en la bitácora del piloto, aquí se muestran los vuelos registrados con la información resumida de cada uno, número de vuelo, ruta, matrícula de la aeronave y fecha para consultar cada vuelo y ver sus detalles y agregar nuevos.

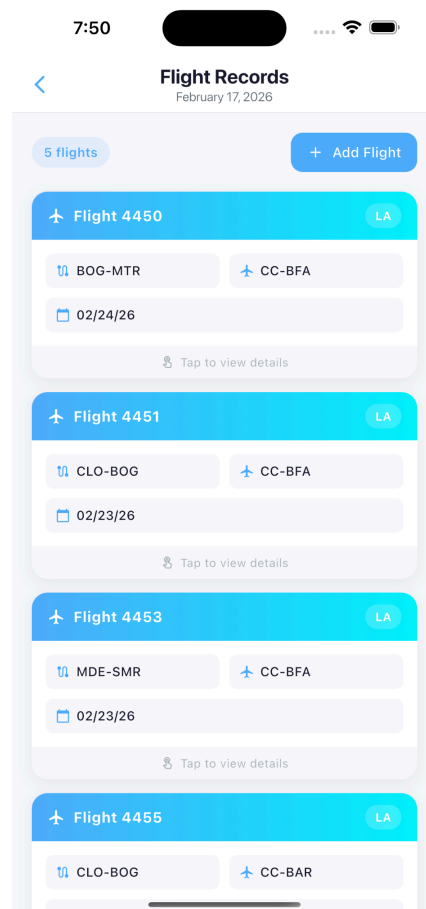


Figura 14. Consultar Vuelos de una bitácora

9.1.15 Consultar la información del vuelo

Pantalla que muestra un vuelo registrado en la bitácora del piloto, mostrando numero de vuelo, la ruta, la matrícula de la aeronave, hora de salida, hora de llegada, cantidad de pasajeros y los tiempos operacionales OUT, OFF, ON e IN los cuales son fundamentales para el cálculo preciso de las horas de vuelo según la normativa aeronáutica junto con el rol de la tripulación, rol de piloto, nombre del compañero, tipo de aproximación y tipo de vuelo además del cálculo de los tiempos de vuelo Air Time y Block Time y permite al piloto ver, editar y eliminar toda la información.

Figura 15: Consultar la información del vuelo

Figura 16: Editar y eliminar información

Figura 15. Consultar la información del vuelo y **Figura 16.** Editar y eliminar información

9.1.16 Consultar las alertas

Pantalla de alertas y resumen de horas de vuelo que muestra el estado de alertas y permite consultar las horas de vuelo con filtros por período: Mensual, Bimestral, Trimestral y Semestral y se muestra completamente el seguimiento periódico de horas.

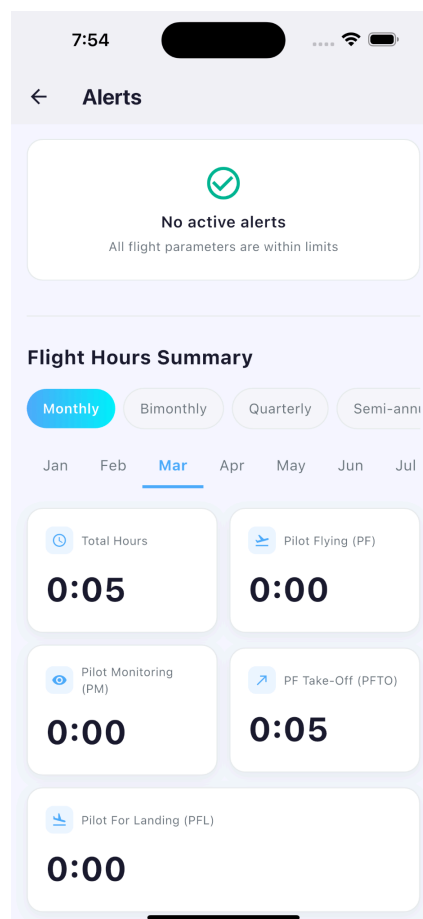


Figura 17. Consultar Alertas

9.2 Perspectiva del administrador

El administrador es el usuario que se encarga de todas las funcionalidades que correspondan con activar, desactivar, lo relacionado a ruta , ruta aerolínea , aerolínea , tipo aeronave y aeropuerto.

9.2.1 Pantalla de inicio

Sección de configuración del sistema que presenta las opciones de gestión disponibles para el administrador: aerolíneas, rutas de vuelo, rutas por aerolínea, aeropuertos, modelos de aeronave, familias de aeronave y fabricantes junto con la razón de que cada una de estas categorías constituye un dato maestro necesario para la generación precisa de la bitácora de vuelo.

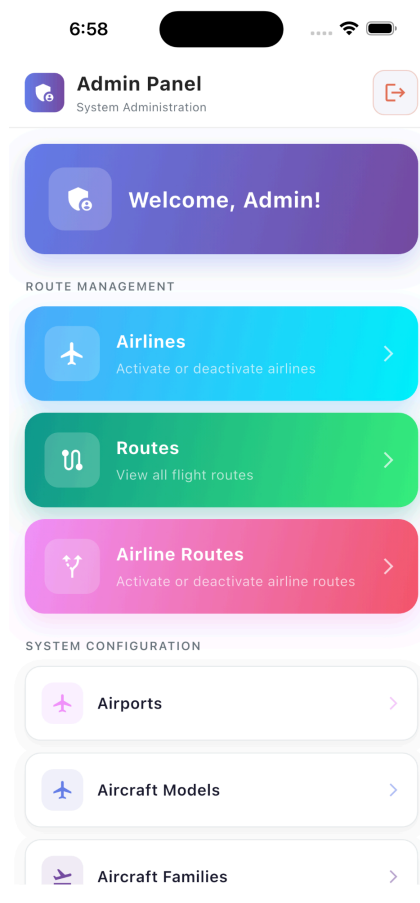


Figura 1. *Pantalla de inicio*

9.2.2 Gestión de aerolíneas

Pantalla que muestra el listado de aerolíneas registradas en el sistema cada una con su código de la aerolínea correspondiente y estado de actividad además de estar diseñada para que el administrador active y desactive cualquiera de ellas.

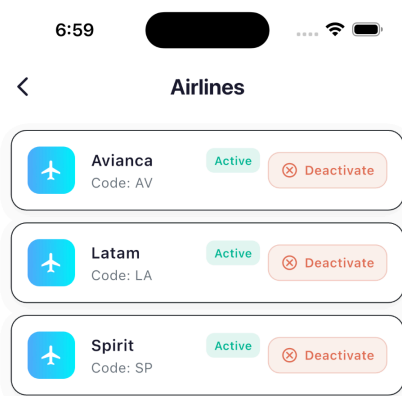


Figura 2. *Gestión de aerolíneas*

9.2.3 Gestión de rutas de aerolínea

Pantalla en la que se muestran las rutas asignadas específicamente a la aerolínea a la que pertenezca el piloto autenticado, mostrando las rutas operativas lo que permite que cada piloto solo vea las rutas correspondientes a su aerolínea al registrar sus vuelos ya que el administrador puede activar y desactivar cualquiera de ellas.

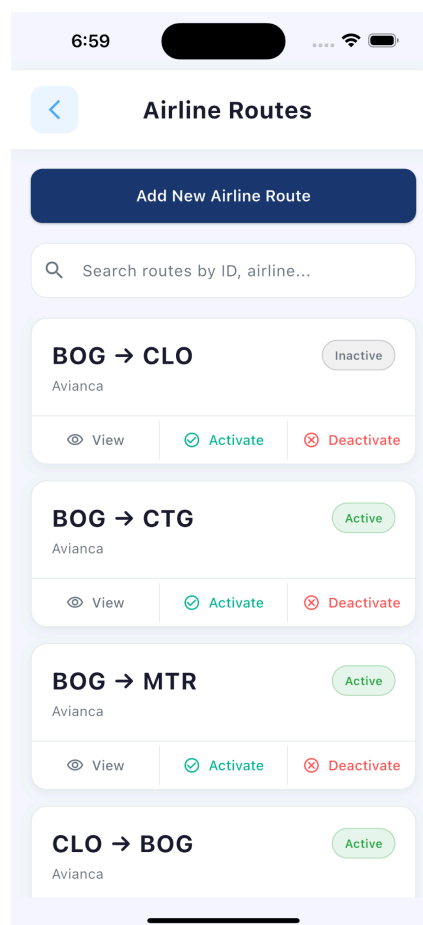


Figura 3. *Gestionar Rutas de la Aerolínea.*

9.2.4 Gestión de rutas

Plantilla que muestra el listado de rutas de vuelo disponibles en el sistema, con información del aeropuerto de origen, aeropuerto de destino y estado de cada una, las cuales son las que los pilotos pueden seleccionar al momento de registrar un vuelo en su bitácora.

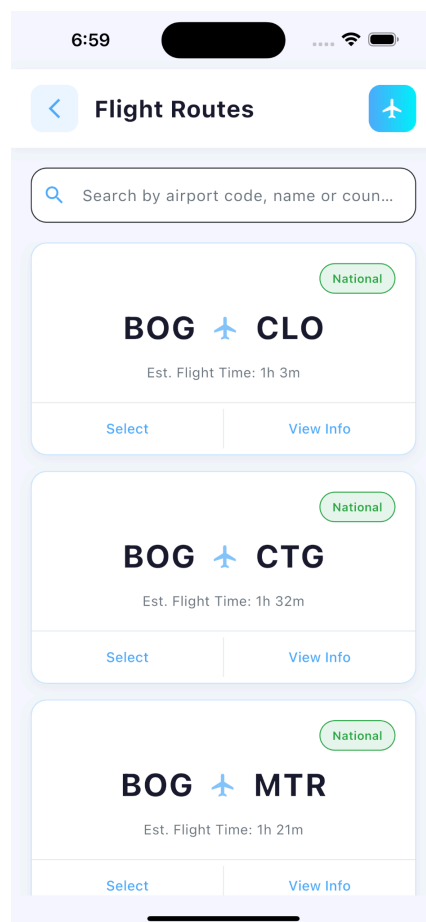


Figura 4. *Gestión Rutas*

9.2.5 Gestión de aeropuertos

Pantalla que muestra el Catálogo de aeropuertos registrados en la plataforma, con sus códigos IATA los cuales conforman los puntos de origen y destino de las rutas que los pilotos utilizan al completar su bitácora diaria.

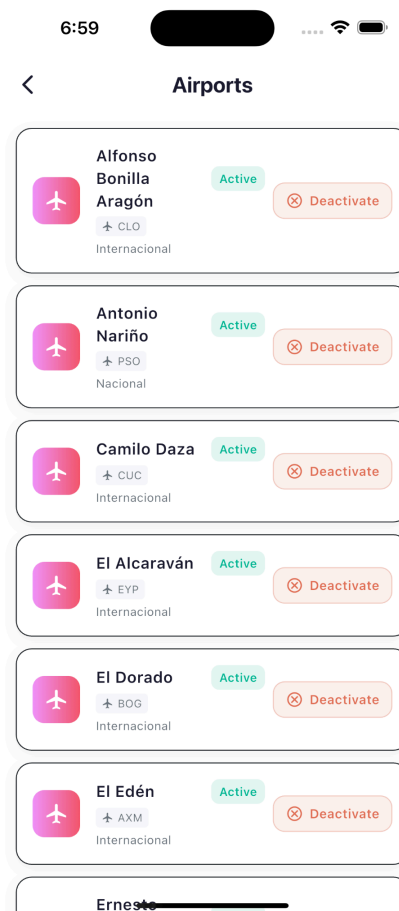


Figura 5. *Gestión Aeropuertos*

9.2.6 Gestión de modelos de aeronave

Plantilla que muestra el listado de modelos de aeronave registrados en el sistema, con su nombre, tipo, familia, motor y estado que el administrador puede activar o desactivar ya que el registro de los modelos de aeronave es esencial para la bitácora de vuelo debido a que cada vuelo debe asociarse a la aeronave específica en la que se realizó.

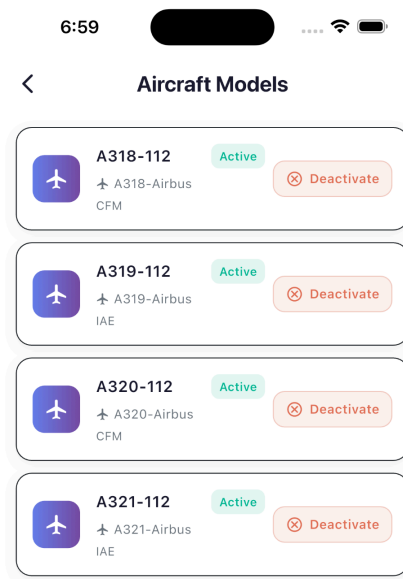


Figura 6. *Gestión Modelos de aeronave*

9.2.7 Gestión de familias de aeronaves

Pantalla que muestra el catálogo de familias de aeronave que agrupa los modelos por tipo de avión y fabricante, la cual es una clasificación que facilita la organización y el filtrado de aeronaves en el sistema.

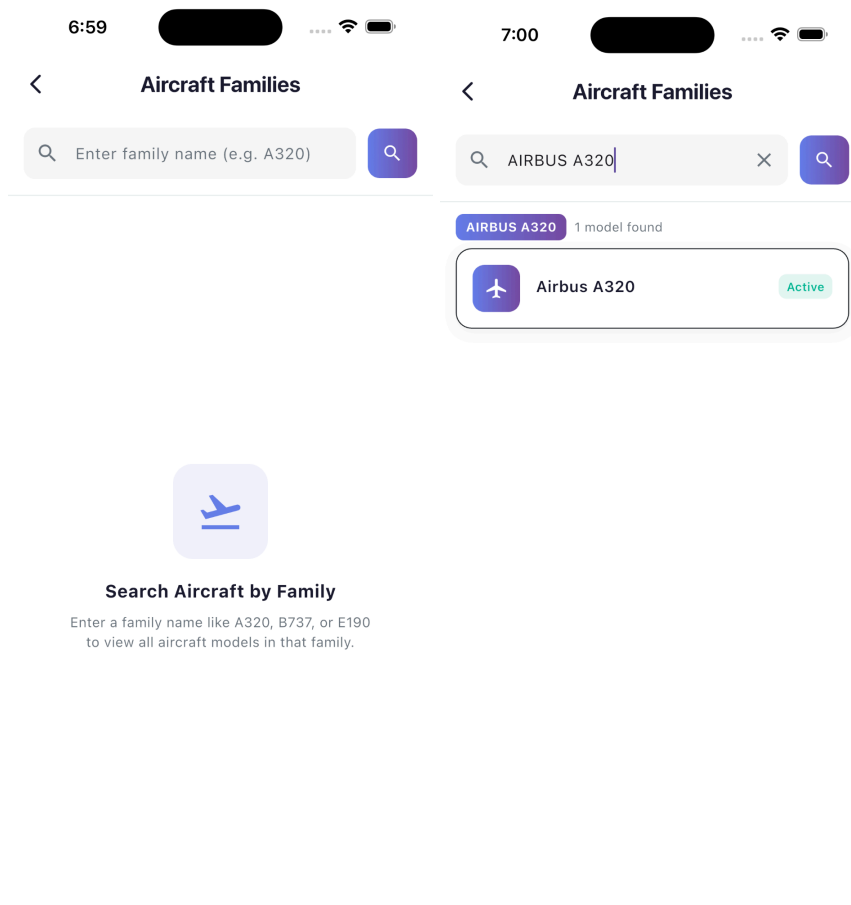


Figura 7 *Gestión de Familias de Aeronaves*

9.2.8 Gestión de fabricantes

Pantalla que muestra el listado de fabricantes de aeronaves registrados en la plataforma como Airbus y Boeing lo cual completa la trazabilidad técnica de las aeronaves utilizadas en los vuelos registrados de la bitácora.



Figura 9. *Gestión de Fabricantes*

10. CONCLUSIONES

Se puede evidenciar que el piloto ya no tiene que hacer mucho esfuerzo para registrar sus horas de vuelo. Con la creación de la bitácora diaria, el registro de fechas, horas, el rol que se ejerce en la cabina y demás información relacionada con sus vuelos, se calculan las horas totales y diferidas entre las distintas actividades, permitiendo las alertas tempranas para prevenir fatiga por exceso de trabajo y el ahorro de dinero o sanciones por parte de la aerolínea.

Un factor a resaltar en cuanto a FlightHours es que el piloto puede hacer seguimiento al mínimo número de aterrizajes que debe de cumplir cada 90 días ejerciendo el rol como PF o PFL, evitando la pérdida de su habilitación la cual afectaría directamente su empleo y su carrera profesional.

Con base en lo que se hizo actualmente, se podría integrar una funcionalidad que le permita al piloto tomar una foto a las horas de vuelo que escribió en el libro físico y que el sistema a partir de esta, extraiga los datos y las registre automáticamente en la plataforma.

Se lograron cumplir todas las historias de usuario con éxito y lo que mencioné en los objetivos, teniendo todo eso en cuenta existe la posibilidad de implementar una capacidad que distinga entre las horas de vuelo diurnas y nocturnas en una sola entrada en el libro de registro del mismo piloto. Esto significaría más granularidad, lo que coincidiría con los requisitos regulatorios que separan los dos tipos de operación.

11. REFERENCIAS BIBLIOGRÁFICAS

1. Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.
2. Brandolini, A. (2021). Introducing EventStorming: An Act of Deliberate Collective Learning. Leanpub.
https://leanpub.com/introducing_eventstorming
3. Patton, J. (2014). User Story Mapping: Discover the Whole Story, Build the Right Product. O'Reilly Media.
4. Wynne, M., & Hellesøy, A. (2017). The Cucumber Book: Behaviour-Driven Development for Testers and Developers (2nd ed.). Pragmatic Bookshelf.
5. Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional.
6. Adzic, G. (2012). Impact Mapping: Making a Big Impact with Software Products and Projects. Provoking Thoughts.
7. Unidad Administrativa Especial de Aeronáutica Civil [UAEAC]. (2023). Reglamentos Aeronáuticos de Colombia (RAC).
<https://www.aerocivil.gov.co/normatividad/reglamentos-aeronauticos>
8. Organización de Aviación Civil Internacional [OACI]. (2006). Anexo 1 — Licencias al personal (10ª ed.). OACI.
9. Fowler, M. (2003, 25 de noviembre). AnemicDomainModel. MartinFowler.com. <https://martinfowler.com/bliki/AnemicDomainModel.html>
10. Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. <https://scrumguides.org/>

11. Driessen, V. (2010, 5 de enero). A successful Git branching model. nvie.com. <https://nvie.com/posts/a-successful-git-branching-model/>

12. Conventional Commits. (s.f.). Conventional Commits: A specification for adding human and machine readable meaning to commit messages. <https://www.conventionalcommits.org/>

13. Fowler, M. (2010, 18 de marzo). Richardson Maturity Model. MartinFowler.com. <https://martinfowler.com/articles/richardsonMaturityModel.html>

14. Logten Pro
<https://coradine.com/about-us/>

15.ForeFlight
<https://foreflight.com/products/foreflight-web/>

16.FlightLogger
<https://flightlogger.net/features/>

17.CrewLounge
<https://crewlounge.aero/about/>

18.ZuluLog
<https://www.zululog.com/>

19 IATA
https://www.iata.org/contentassets/a8e49941e8824a058fee3f5ae0c005d9/safety-report-executive-summary-and-safety-overview_2025.pdf

20.INEC
<https://aviacioninec.com/el-factor-humano-en-la-aviacion/>

21.Opinion
<https://www.opinion.com.bo/articulo/informe-especial/80-ciento-accidentes-avi-oacut-e-n-ocurre-fallas-humanas/20161210234000671993.html>

22 simutech

<https://www.simutech.com.au/post/addressing-pilot-frustrations-with-flight-simulator-training>

12. GLOSARIO

- AL(AUTOLAND):La aeronave aterriza solo en condiciones de baja visibilidad.
- AR(Authorization required): proceso de aproximación por instrumentos que requieren autorización para ser ejecutados.
- AIR:Tiempo desde el despegue hasta el aterrizaje realizado por la aeronave.
- ACUM MES:Tiempo Total a pagar debido al total de las horas de vuelo de la jornada.
- BP:Código de identidad del piloto de una aerolínea.
- BLOCK:Tiempo desde el remolque hasta el parqueo de la aeronave.
- CMD: Capitán con el que voló el piloto el vuelo.
- CAT(aproximación de categoría):Aproximación cuando hay condiciones de baja visibilidad durante el vuelo.
- DIA DE DESCANSO: Día libre para el piloto.

- DIA B(Dia Blanco):Solo un día de descanso que no cuenta entre los 9 reglamentarios asignado al piloto.
- DUTY:Tiempo de servicio. Tiempo transcurrido entre la hora de presentación y el parqueo del último vuelo que realizó el piloto .
- Est Block:Tiempo de vuelo estimado de la aeronave.
- HLC:Hora Local en la que se realiza el vuelo.
- IN:Momento en que la aeronave aterriza y desembarca en una puerta asignada.
- OUT:inicio tiempo de vuelo.
- OFF:Hora de despegue de la aeronave.
- ON:Hora de aterrizaje de la aeronave.
- PF(Pilot Flying):el piloto que realizó el vuelo.
- PFTO(Pilot Flying For Take OFF): el piloto que realizó el despegue del vuelo.
- PM(Pilot Monitoring): El piloto que monitoreo los procedimientos de la aeronave.
- PFL(Pilot Flight For Landing):El piloto solo realizó el aterrizaje del vuelo.
- PAX: Cantidad de pasajeros del vuelo.
- PRES:Hora de presentación del piloto en el aeropuerto.
- REAL: Hora a la que terminó de volar.
- TM: Tripulación de mando, pilotos.
- TC:Tripulación de cabina,auxiliares de vuelo.
- TOTAL:Total de horas de DUTY.
- UTC(Universal Time Coordinated):Hora coordinada. hora en el meridiano de Greenwich.
- SOP(Standard Operating Procedure): Procedimiento Operativo Estándar. Son instrucciones escritas, paso a paso diseñadas para garantizar la seguridad, la

eficiencia y la coherencia en las operaciones de la cabina de vuelo, la asistencia en tierra y el mantenimiento; proporcionan un modelo mental compartido para los miembros de la tripulación, detallando quién hace qué, cuándo y cómo a la hora de realizar las tareas.

- **Aeropuerto:** Todo aeródromo especialmente equipado y usado regularmente para pasajeros y/o carga y que, a juicio de la UAEAC(Unidad Administrativa Especial de Aeronáutica Civil), posee instalaciones y servicios de infraestructura aeronáutica suficientes para ser operado en la aviación civil.
- **Aeropuerto internacional:** Todo aeropuerto que cuenta con facilidades adecuadas al tráfico aéreo internacional y que el estado contratante en cuyo territorio está situado, designa como aeropuerto de entrada o salida para el tránsito aéreo internacional; en el cual se llevan a cabo trámites de aduana, migración, sanidad, cuarentena agrícola y demás procedimientos similares, requeridos.
- **Miembro de la tripulación de vuelo:** Tripulante, personal licenciado, piloto, copiloto o ingeniero asignado a la cabina de mando esencial para la operación de una aeronave durante el tiempo de vuelo, responsable de la navegación, control técnico y toma de decisiones durante todas las fases del vuelo desde la presentación hasta el post-vuelo.

- **Tripulación sencilla:** Es el número mínimo de tripulantes técnicos, pilotos y de cabina, auxiliares de vuelo, requeridos por las autoridades aeronáuticas y el fabricante para operar un vuelo específico.
- **Tripulante:** Persona titular de la correspondiente licencia, a quien el explotador de una aeronave asigna obligaciones o tareas que ha de cumplir a bordo durante el tiempo de vuelo de la aeronave.
- **Angular JS:** Es una biblioteca de JavaScript que permite crear aplicaciones de una sola página, lo que quiere decir que el navegador nunca recarga la página solo carga el nuevo contenido.
- **Api(Application Programming Interface):** Es una interfaz que permite conectar aplicaciones con diferentes servicios u otras aplicaciones.
- **Aplicación:** Es un software que tiene un propósito específico.
- **Apache HTTPd:** Servidor Web Libre. Dicho servidor es altamente flexible y actualmente el 60% de los sitios web funcionan con Apache.
- **API Rest:** Tipo de desarrollo Backend que implementa un servicio web que no tiene estado.
- **Backend:** Tipo de desarrollo web programado para ejecutarse principalmente en el servidor. Normalmente accediendo a una base de datos.
- **Base de datos:** Es un conjunto de datos (direcciones, productos, ubicaciones geográficas, contactos, etc.) organizados en un tipo de estructura que permite su consulta.

- **Biblioteca:** Es un conjunto de funciones, recursos y demás elementos que facilitan la programación de una aplicación.
- **Bug:** Es la forma común de llamarle a los errores o defectos encontrados en las aplicaciones.
- **Código abierto:** Es una tendencia en la que el código del software es público, entre sus características, cualquier persona puede usarlo, compartirlo, modificarlo o crear una variante del software inicial.
- **Css(Cascading Style Sheets):** Hojas de estilo en cascada.
- **Dominio:** Es el nombre por el cual se identifica un sitio web o una red.
- **Dirección IP:** Identificativo único de una interfaz de red basado en el Protocolo IP. Normalmente asociada a un nombre de dominio. Necesaria para crear una conexión Socket/IP.
- **Dominio:** Nombre registrado y asociado a una compañía que permite asociar nombres de servidores con direcciones IP.
- **Desarrollo Multiplataforma:** Consiste en crear aplicaciones móviles que puedan ejecutarse en varias plataformas lo cual se logra mediante el uso de marcos como React Native,Flutter y más que existen en el mercado, las cuales permiten a los desarrolladores escribir una única base de código que se puede implementar en plataformas como iOS,Android y web, la finalidad es facilitar el desarrollo de aplicaciones móviles con una base de código compartida en lugar de hacerlas por separado para cada plataforma.
- **Desarrollo de Aplicaciones Móviles:** El hecho de crear aplicaciones móviles para la aeronáutica implica considerar la accesibilidad y la seguridad además de la eficiencia en la transmisión de datos dando por hecho que estas aplicaciones pueden ser herramientas valiosas para el monitoreo remoto de la

aeronave, la gestión de itinerarios y la comunicación instantánea entre el personal.

- **Extensión o plugin Web:** Extensión de un navegador web que permite añadir una funcionalidad a un navegador web.
- **Emulador:** Es una herramienta o programa que permite replicar un sistema informático, incluido su hardware, sistema operativo y funcionalidades esto permite probar y ejecutar aplicaciones en un entorno que imita el sistema de destino.
- **Etiqueta o Tag:** Unidad mínima de representación en el Lenguaje HTML. Permite definir párrafos, tablas, enlaces, imágenes, etc.
- **Frontend:** Tipo de desarrollo web programado fundamentalmente en JavaScript y ejecutado en el navegador web.
- **HTML(Hypertext Markup Language):** Es un lenguaje que permite definir el contenido para que el cliente web permita visualizar la información que contiene.
- **HTTP(Hypertext Transfer Protocol):** Permite la comunicación de Internet, protegiendo la integridad y la confidencialidad de los datos de los usuarios entre sus ordenadores y un sitio web.
- **Introducción a la Ingeniería de Software Móvil:** La ingeniería de software móvil, está enfocada en el diseño y desarrollo de aplicaciones para dispositivos móviles. En el ámbito aeronáutico, las aplicaciones móviles facilitan la comunicación entre la tripulación y lo relacionado con el seguimiento en tiempo real y la notificación de eventos críticos.

- **JavaScript:** Lenguaje de programación orientado a objetos y eventos que permite en la parte cliente, procesar y realizar cambios en la presentación dentro del navegador.
- **Java:** Es un lenguaje de programación multipropósito, interpretado que se ejecuta en una máquina virtual.
- **JQuery:** Es una biblioteca de JavaScript que tiene como finalidad simplificar el desarrollo de páginas web interactivas.
- **JSON(Javascript Object Notation):** Es un formato para almacenar, transmitir o representar datos con estructura.
- **Lenguaje de Programación:** Es un conjunto de palabras, expresiones y símbolos que permiten escribir instrucciones que las máquinas pueden interpretar y ejecutar.
- **Linux:** Sistema operativo de código fuente abierto basado en Unix inventado por Linus Torvalds, el cual en los últimos años ha tomado auge en el sector web.
- **MySQL:** SGDB Creado por MySql AB.
- **Módulo de Apache:** Conjunto de funcionalidades agrupadas que permiten extender el tratamiento de la información por parte del servidor Web Apache.
- **Métodos HTTP:** Verbos que identifican el tipo de petición HTTP que se quiere realizar.
- **Método GET:** Tipo de petición que quiere solicitar información al servidor web. También permite pasar parámetros por la URL, pero de manera visible, no como POST.

- **Método POST:** Tipo de petición que permite enviar datos al servidor de manera transparente. Forma de paso de parámetros mediante formulario (normalmente).
- **Navegador Web:** Cliente que permite consultar a Servidores Web y presentar la información HTML.
- **PHP(Hypertext PreProcessor):** Es un lenguaje de programación creado por Rasmus Lerdorf usado generalmente para la creación de contenido para sitios web.
- **Puerto IP:** Número identificativo de una puerta abierta en un servidor.
- **PostgreSQL:** SGBD profesional libre de alto rendimiento y alta capacidad.
- **Protocolo:** es la manera o el lenguaje por el cual se comunican las máquinas.
- **Página Web:** Unidad de contenido mínima de un servidor web.
- **SGBD(Sistema de Gestión de Bases de Datos):** Son un tipo de software muy específico, dedicado a servir de interfaz entre la base de datos, el usuario y las aplicaciones que la utilizan. Se compone de un lenguaje de definición de datos, de un lenguaje de manipulación de datos y de un lenguaje de consulta.
- **SQL(Structured Query Language):** Es un lenguaje declarativo de acceso a bases de datos relacionales que permite especificar diversos tipos de operaciones sobre las mismas.
- **Servicio Web:** Aplicación web Backend.
- **SOAP(Simple Object Access Protocol):** Protocolo de servicio web backend estructurado con peticiones post y XML.
- **Servidor:** Es un computador que presta algún tipo de servicio.
- **Sitio web:** Conjunto de reglas del Servidor Web y las páginas Web que lo componen.

- **XML(eXtensible Markup Language):** Formato de fichero con etiquetas extensibles.

13. LISTA DE ANEXOS

el nombre “<Repositorio Institucional>” hace referencia a la ubicación donde oficialmente la universidad ha dispuesto en custodia en los artefactos, fruto de este trabajo de grado de acuerdo a lo especificado en cada uno de los anexos

Tabla 2

Anexos de artefactos estratégicos del proyecto FlightHours

N°	Nombre	Ruta
1	Visión	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\1. Visión
2	Impact Mapping	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\2. Mapa de Impacto
3	Modelo de dominio	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\3. Modelo de dominio\Requisitos de información
4	Event Storming	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\4. Event Storming
5	Mapa de historias de usuario	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\5. Mapa de historias de usuario
6	Requerimientos Funcionales	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\Funcionales
7	Requerimientos Funcionales	No <Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\No funcionales
8	Reglas de Negocio	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\Reglas de negocio

Tabla 3*Anexos de artefactos técnicos del proyecto FlightHours*

N°	Nombre	Ruta
9	Atributos de calidad	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\1. Atributos de calidad
10	Funcionalidades críticas	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\2. Funcionalidades críticas
11	Restricciones técnicas	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\3. Restricciones técnicas
12	Restricciones de negocio	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\4. Restricciones de negocio
13	Spikes (Pruebas de concepto)	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\2. Spikes (Pruebas de concepto)
14	Alternativas de solución	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\3. Alternativas de solución
15	Arquetipo de solución	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\4. Arquetipo de solución
16	Plataforma tecnológica	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\5. Plataforma tecnológica
17	Diagrama de clases	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\1. Diagrama de clases
18	Diagrama de componentes	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\2. Diagrama de componentes

19	Diagrama de paquetes	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\3. Diagrama de paquetes
20	Diagramas de secuencia	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\4. Diagrama de secuencia
21	Diagrama de despliegue	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\5. Diagrama de despliegue
22	Modelo Entidad Relación	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\6. Modelo entidad relación

Tabla 4*Anexos de construcción del proyecto FlightHours*

N°	Nombre	Ruta
23	Análisis estático de código	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\1. Calidad de código\Análisis_Estático_Flight Hours.xlsx
24	Informe de pruebas de carga	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\1. Calidad de código\Análisis_Test_Carga_FlightHours.xlsx
25	Código fuente API	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\2. Código fuente\api
26	Configuración Backend	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\2. Código fuente\api\backend-config\api-flighthours-0.17.5.zip
27	Código fuente Frontend	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\2. Código fuente\app\Flighthours_frontend-0.18.0.zip
28	CI/CD Pipeline (Devops)	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\3. Devops\Pipeline_CICD_FlightHours.xlsx
29	Patrones de Diseño	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\3.Construcción\4. Patrones de Diseño\Patrones_Diseno_FlightHours.xlsx

Tabla 5*Anexos de puesta en marcha del proyecto FlightHours*

Nº	Nombre	Ruta
30	Manual de instalación de bases de datos	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\Manual_Instalacion_BD_FlightHours.xlsx
31	Manual de instalación	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\Manual_Instalacion_FlightHours.xlsx
32	Manual de despliegue	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\Manual_Despliegue_FlightHours.xlsx
33	Dump base de datos de negocio	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\data-flighthours.sql
34	Dump base de datos Keycloak	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\data-keycloak.sql
35	Schema base de datos de negocio	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\schema-flighthours.sql
36	Schema base de datos Keycloak	<Repositorio institucional>\FlightHours\1.Artefactos proyecto\4. Puesta en marcha\schema-keycloak.sql